phonokit

v 0.6.0

A toolkit to create phonological representations in Typst

Guilherme D. Garcia

Pavillon Charles-De Koninck, 1030, Bureau DKN-3267

Département de langues, linguistique et traduction, UNIVERSITÉ LAVAL

Avenue des Sciences-Humaines, Québec QC, Canada

gdgarcia.ca guilhermegarcia guilherme.garcia@lli.ulaval.ca

DOI 10.5281/zenodo.17971032

Abstract

Creating high-quality phonological representations in academic documents is often time-consuming and requires juggling multiple tools and LaTeX packages (most notably `TikZ`). This vignette introduces **phonokit**, an open-source Typst package designed to streamline the creation of phonological structures while maintaining typographical precision. The package provides intuitive functions for IPA transcription (supporting `tipa` input conventions), IPA vowel charts and consonant tables with built-in language inventories and flexible arrows, prosodic representations (syllables, moras, feet, prosodic words, metrical grids), sonority profiles, autosegmental phonology (feature spreading, tone, delinking), multi-tier representations for complex non-linear structures such as Government Phonology and CV phonology, feature geometry, SPE feature matrices, Optimality Theory tableaux, Harmonic Grammar, Noisy Harmonic Grammar, Maximum Entropy grammars, and Hasse diagrams for constraint rankings. The package also includes numbered linguistic examples and a collection of helper symbols for Greek letters and arrows. All functions prioritize minimal input syntax while automatically handling typographical challenges such as dynamic spacing and proper alignment. The font used by the package can be customized globally. **phonokit** aims to reduce cognitive load in document preparation, allowing phonologists to focus on content rather than formatting. The package is freely available through Typst's package repository and is under active development.

Questions, suggestions, bugs

Any questions, comments or suggestions should be posted to the repository below (issues). All the bugs you find will help improve the package! Simply open an issue.

- `gdgarcia.ca/phonokit`
- `guilhermegarcia/phonokit`
- `guilhermegarcia/phonokit/discussions`
- `guilhermegarcia/phonokit/issues`

Note on development

phono*kit* was conceived, designed, and directed by the author. The implementation was produced with the assistance of Claude (Anthropic). All linguistic decisions, function design, and package architecture are the author's.

Version history: what's new?

`0.6.0` - New function `#vowels-3d()` : rotatable 3D vowel space
`0.5.20` - Better spacing in prosody-related functions
`0.5.19` - General bug fixes
`0.5.18` - German vowels: adds /ə/
`0.5.17` - Portuguese consonants: /ɾ/ vs. /ɾ/, no affricates
`0.5.16` - `circle-size` argument in `#vowels()` ; inventory check: `#vowels()` & `#consonants()`
`0.5.15` - Alignment is now more accurate for `#consonants()`
`0.5.14` - General improvements and bug fixes
`0.5.13` - `#vowels()` and `#consonants()` now accept the `phoneme-colors`
`0.5.12` - Several bug fixes
`0.5.11` - `#vowels()` & `#consonants()` now accept a `lang` -only call; laterals added to `#geom()`
`0.5.10` - Updated CeTZ dependency to 0.5.2
`0.5.9` - README updates and documentation refresh
`0.5.8` - Helper for coordinates in `#multi-tier()` ; updated dependencies
`0.5.7` - Automatic node names for `#multi-tier()` ; coordinate references still supported
`0.5.6` - VOT function to create pedagogical VOT timelines
`0.5.5` - Better alignment for numbered examples; vowel dispersion; nasal vowels; UI language support
`0.5.4` - Minor bug fixes in tableaux after recent changes to 0-index
`0.5.3` - Custom spacing for prosody; improved IPA coverage; more flexible tableaux
`0.5.2` - Improved numbered examples with simpler code
`0.5.1` - Prosodic representation in tableaux, candidate letters, gloss support, 0-indexed dashed lines
`0.5.0` - Feature geometry with support for arrows, delinking, and highlights
`0.4.6` - Inline ToBI function for intonational labels and titles for numbered examples
`0.4.5` - Vowel trapezoids accept arrows and shifted vowels
`0.4.1` - Consonants table has abbreviation argument; alignment fix in `#consonants()`

`0.4.0` - Multi-tier representations; sorted MaxEnt candidates; helper functions for Greek letters

`0.3.7` - Font and prosodic symbols can now be chosen; aesthetic improvements to prosodic functions

`0.3.6` - Improved IPA symbols, distances in `#syllable()`, and `README.md`

`0.3.5` - Numbered examples; minor improvements in IPA symbols and several fixes in documentation

`0.3.0` - Autosegmental phonology

`0.2.0` - Sonority, tableaux and metrical grids

`0.0.1` - Initial release with prosodic representations and IPA module

Table of contents

1. Introduction 5
    1.1. Installation 5
    1.2. Font 5
2. IPA module 6
    2.1. Phonemic transcription 6
    2.2. Phonemic inventories 6
        2.2.1. Consonants 6
        2.2.2. Vowels 8
        2.2.3. 3D vowel space 11
3. Phonetics 13
    3.1. Sound shifts 13
    3.2. Voice Onset Time 14
    3.3. Vowel dispersion 15
4. SPE 16
5. Prosody module 18
    5.1. Sonority 18
    5.2. Syllables 18
    5.3. Feet 19
    5.4. Prosodic words 21
    5.5. Metrical grid 24
6. Autosegmental phonology 24
    6.1. Tones 25
    6.2. Prosody with ToBI 28
    6.3. Multi-tier representations 29
        6.3.1. Help with labels 33
7. Feature geometry 34
    7.1. Single trees 35
    7.2. Multiple trees and processes 37
8. Optimality theory 39
9. Maximum Entropy grammars 42
10. Numbered examples 44
11. Final remarks 47

References 48

Appendix 50

1. Introduction

This vignette introduces **phonokit** and its functions. If you haven't used Typst yet and want to follow along, go to Typst.app to use their online editor. You may want to check their own tutorial too (I have an introductory YouTube series here). This is **not** an introduction to Typst, but see Section D, Section E and Section F in the appendix for some useful info. The GitHub repository for the package can be found at `guilhermegarcia/phonokit`. Comments and suggestions are welcome, as are bug reports (open an issue in the package's repository). The main goals of **phonokit** are to MINIMIZE EFFORT and MAXIMIZE QUALITY.

1.1. Installation

Typst packages are always loaded the same way: using the `#import` function at the top of your `typ` document, as shown in Code 1. Replace `X.X.X` with the version you wish to import. The `*` simply states that we want to import all functions from the package in question. See package's page on Typst's website here.

```
#import "@preview/phonokit:X.X.X": *
```

Code 1: Loading a package in Typst using the official repository

Alternatively, if you want the most up-to-date version and this version is ahead of the published version, download/clone the repository here and load the package locally. This is shown in Code 2. You simply need to import the `lib.typ` file, which is present in any package.

```
#import "phonokit/lib.typ": *
```

Code 2: Loading a package in Typst using a local package

You may need to create a symlink depending on how you structure your files. See Section E for more information on Typst packages in general, as they work slightly differently from what you may be used to if you use LaTeX.

1.2. Font

All functions in **phonokit** require the Charis font (SIL International, 2025) to work as intended out of the box. Thus, if you don't have the font, you will need to install it (unless you're ok with the fallback option in Typst). However, as of version `0.3.7`, the user can set a global font to be used by the package throughout the document. Code 3 shows how to set New Computer Modern as the main font for the package.

```
#import "@preview/phonokit:0.6.0": *
#phonokit-init(font: "New Computer Modern") // ← add to the top of your document
```

Code 3: Choosing a different font

2. IPA module

IPA transcription is likely the most commonly used feature when typesetting documents in phonology. **phono*kit*** accomplishes that with the `#ipa()` function, which takes a string as input. Crucially, the function uses the familiar `tipa` input (Rei, 1996), with a few exceptions (e.g., secondary stress is represented by a comma `,`, not by two double quotes `""`).

2.1. Phonemic transcription

As can be seen in Code 4, symbols introduced by two backslashes `\\` must not have adjacent characters. For archiphonemes, you can use `\\` followed by a capital letter of your choice. This is nice because you don't need to leave the function to render an archiphoneme, so the font will be automatically consistent across phonemes and archiphonemes. Thus, while `#ipa("N")` maps to "ŋ", `#ipa("\\N")` maps to "N" — see examples.

`#ipa("DIs \\s Iz \\s @ \\s sEn.t@ns")`	ðɪs ɪz ə sɛn.təns	*This is a sentence*
`#ipa("N \\N R \\R \\I I Z \\Z")`	ŋNɾRIɪʒZ	*Escaped characters*
`#ipa("p \\h I k \\* \\s \\t tS \\ae t \\s p \\r l iz")`	pʰɪkˈ t͡ʃæt pl̥iz	*Pick, chat, please*
`#ipa("'lIt \\v l \\s 'b2R \\schwar ,flaI")`	ˈlɪtl̩ ˈbʌɾɚˌflaɪ	*Little butterfly*

Code 4: Transcriptions using `#ipa()`

See Section A for a reference table with the main IPA symbols available. For a more comprehensive list, visit the package's repository and download `ipa_reference.pdf` inside the `extras` directory.

2.2. Phonemic inventories

2.2.1. Consonants

Two additional functions allow users to quickly create consonant tables and vowel trapezoids given a string of phonemes. Figure 1 shows the consonant inventory for Italian, for example. The function mirrors the pulmonic consonants table in the IPA chart with some minor changes. For example, affricates are shown when `affricates: true`, and /w/ is shown in the approximant row under both bilabial and velar columns (when /ɥ/ is not present, in which case /w/ appears only under bilabial). The argument `abbreviate: true` shortens labels for both rows and columns.

Aspirated consonants are shown when `aspirated: true`. In cases where neither `affricates` nor `aspirated` are set to `true`, the function will omit both groups and fewer rows will be printed.

The user can either input a language[1] (see caption of Figure 1) or a string of consonants to create a custom inventory (Figure 2) — the input follows the same format used by the `#ipa()` function discussed in Section 2.1, so `#ipa("\\*r")` generates "ɾ". Note, however, that both affricates and aspirated consonants require curly braces around them as well as `affricates: true` and `aspirated: true`, as shown in the caption of Figure 2. As of version `0.5.13`, individual consonant symbols can be colored with `phoneme-colors`, for example

[1] Available languages: Arabic, English, French, German, Italian, Japanese, Portuguese, Russian and Spanish (all language names are lowercase in the function). You can also use `all` to display all consonants. The same applies to the function `#vowels()`.

`#consonants("ptk", phoneme-colors: ("p": red, "t": blue))`. Finally, the function also allows for flexible sizing with the `scale` argument.

	Bilab	Labdent	Dent	Alv	Postalv	Retro	Pal	Vel	Uvu	Phar	Glot
Plos	p b			t d				k g			
Nas	m			n			ɲ				
Trill				r							
Tap/Flap											
Fric		f v		s z	ʃ						
Affr				ts dz	tʃ dʒ						
Lat fric											
Approx							j				
Lat approx				l			ʎ				

Figure 1: Italian consonants - `#consonants("italian", affricates: true, abbreviate: true)`

	Bilabial	Labiodental	Dental	Alveolar	Postalveolar	Retroflex	Palatal	Velar	Uvular	Pharyngeal	Glottal
Plosive	p			t				g			
Plosive (aspirated)								k^h			
Nasal											
Trill											
Tap or Flap											
Fricative				s	ʃ						
Affricate				ts	tʃ						
Affricate (aspirated)											
Lateral fricative											
Approximant				ɹ							
Lateral approximant											

Figure 2: `#consonants("ts{ts}psS \\*r g{tS} {k \\h}", affricates: true, aspirated: true)`

As of version 0.5.3, you can delete rows or columns to create a more minimal table. The relevant arguments are `delete-cols` and `delete-rows`. Both accept an array of positions (starting from 0). For example, you could create a minimal table for English consonants, shown in Figure 3. You can also use `simplify: true` to remove all rows and columns without consonants in them.

	Bilabial	Labiodental	Dental	Alveolar	Postalveolar	Palatal	Velar	Glottal
Plosive	p b			t d			k g	
Nasal	m			n			ŋ	
Fricative		f v	θ ð	s z	ʃ ʒ			h
Affricate					tʃ dʒ			
Approximant	w			ɹ		j	w	
Lateral approximant				l				

Figure 3: Concise consonant table for English

```
  #consonants("english", affricates: true, delete-cols: (5, 8, 9), delete-rows: (2, 3, 5))

// Alternatively, use the simplify argument:
  #consonants("english", affricates: true, simplify: true)
```

Code 5: Code to generate Figure 3

Certain functions have a `ui-lang` parameter for localization. French and Portuguese are supported (defaults to English). This parameter is also available in the functions `#feat-matrix()` (Section 4), `#geom()` and `#geom-group()` (Section 7). Figure 4, for example, is set to French. The same figure also uses the argument `phoneme-colors` (the same argument is also available for `#vowels()`, discussed below).

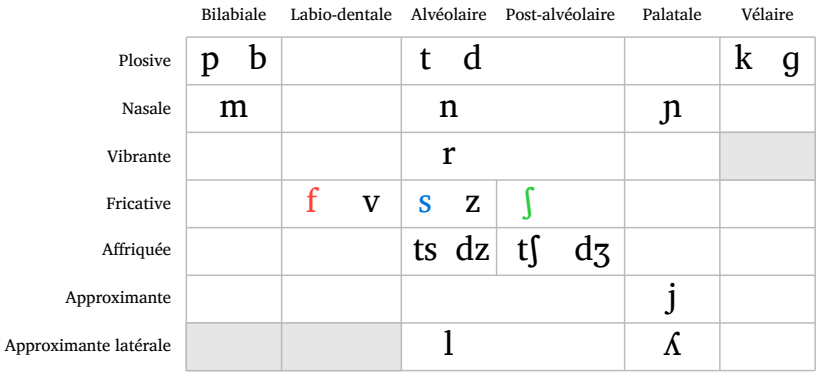


Figure 4: Concise consonant table for Italian in a French document

```
#consonants("italian", affricates: true,
simplify: true,
ui-lang: "fr",
phoneme-colors: (
  "f": red,
  "s": blue,
  "S": green,
))
```

Code 6: Code to generate Figure 4

2.2.2. Vowels

Both `vowels()` and `consonants()` look up preset names before interpreting custom inventories. Thus, `vowels("english")` uses a preset and `vowels("aeiou")` accepts a custom inventory, but `vowels("indonesian")` displays an error instead of a chart: it is not a preset and contains unsupported vowel symbols. The same validation applies to consonant inventories, including braced sequences. Errors list available presets and guidance for custom inventories. Validation follows tipa conversion and preserves supported nasal vowels and whitespace between symbols.

Use `lang:` when you intend a preset, for example `vowels(lang: "english")`. Unknown explicit language names also display an error instead of a chart. The rest of the document continues to compile. A positional name consisting entirely of supported inventory symbols remains indistinguishable from a custom inventory.

Besides the function `#consonants()`, the package has a function to print vowel inventories. The function `#vowels()` also accepts either a pre-defined language or a string as input. Figure 5 and Figure 6 show the inventories for English and French, respectively. The argument `scale` is also available here, so the user can adjust the size of the trapezoid as needed. If desired, the function can also add schematic nasalized copies with `nasals: true`. For preset languages, this currently adds only the French nasal vowels. For custom strings, only vowels that are explicitly nasalized in the input are shown in the nasal layer, as in `#vowels("aãioõu", nasals: true)`. These nasal positions are illustrative only and are simply offset slightly from the corresponding oral vowels to avoid overlap. As with `#consonants()`, individual vowel symbols can be colored with `phoneme-colors`, for example `#vowels("aeiou", phoneme-colors: ("i": red, "u": blue))`.

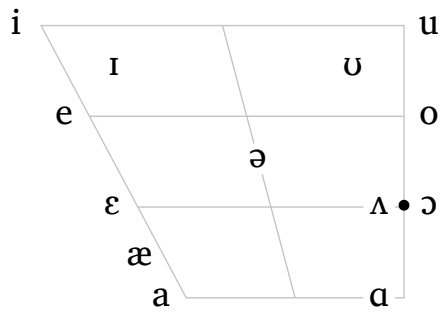


Figure 5: `#vowels("english", scale: 0.6)`

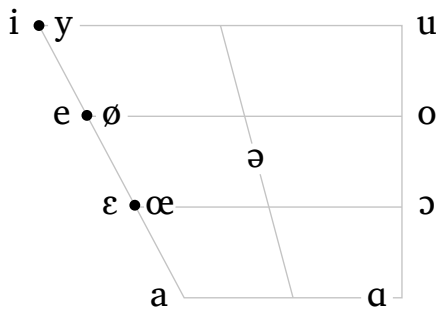


Figure 6: `#vowels("french", scale: 0.6)`

As of version `0.4.5`, the function `#vowels()` accepts a range of additional (optional) arguments to include arrows, shifted vowels, and highlights to a trapezoid. Figure 7 illustrates some of the available arguments to create a trapezoid showing diphthongs in North American English. The arguments are intuitive and self-explanatory: you can use `arrows` to draw arrows between any pair of vowels in the trapezoid. You can also choose if you want arrows to be curved (`curved: true`) or dashed (`arrow-style: dashed`). And you can specify the color for the arrows (`arrow-color`), which is set to a light blue in Figure 7. Highlighted vowels (`highlight`, `highlight-color`) sit on a colored circle, which you can resize with `circle-size` (default: `1`) if it looks too tight or too loose for your font.

```
#vowels(
    "english",
    arrows: (
      ("a", "U"),
      ("a", "I"),
      ("e", "I"),
      ("O", "I"),
      ("o", "U"),
    ),
    arrow-color: blue.lighten(60%),
    curved: true,
    highlight: ("a", "e", "o", "O"),
    highlight-color: blue.lighten(80%),
    phoneme-colors: (
      "i": red, "u": orange
    ),
  )
```

Code 7: Code to generate Figure 7

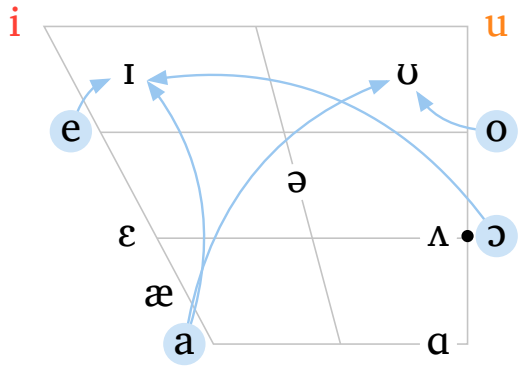


Figure 7: Diphthongs in North American English

The function `#vowels()` places vowels in predetermined locations in the trapezoid, as expected. However, as of version `0.4.5`, you can also shift vowels, which allows for a much higher degree of flexibility when illustrating dialectal variation, for example. The argument `shift`, shown in the code next to Figure 8, allows you to specify a vowel as well as a shift from its original position. Shifted vowels can then be independently targeted by the `highlight` argument, just like any other vowel in the trapezoid. In addition, you can customize arrows to target shifted vowels by using the same shifted values you defined for each vowel. Finally, shifted vowels can have their color and size adjusted with `shift-color` and `shift-size`, respectively (but note that all shifted vowels will share the same color and size). In the example shown in Figure 8, a custom vowel inventory is illustrated with three shifted vowels (in dark blue). The trapezoid also shows how to change the color and style of arrows.

As is often the case, a figure can quickly become too crowded. Once we start adding arrows and shifted vowels, a trapezoid may be less clear given the amount of information displayed. Figure 8 (bottom) shows how you can simplify the representation by removing grid lines (`rows` and `cols`). Last but not least, notice that the trapezoid at the bottom in Figure 8 is scaled down to `0.4` (by default, `#vowels()` uses `scale: 0.7`). Everything in the trapezoid scales down or up accordingly.

```
#vowels(
    "aeiouIU",
    arrows: (
      ("a", ("U", -0.6, -0.3)),
      ("a", ("U", -0.3, -0.7)),
    ),
    arrow-color: blue.lighten(40%),
    arrow-style: "dashed",
    curved: true,
    shift: (
      ("a", 0.6, 0.3),
      ("U", -0.6, -0.3),
      ("U", -0.3, -0.7),
    ),
    shift-size: 1.5em,
    shift-color: blue.darken(20%),
    highlight: ("a", ("U", -0.6, -0.3)),
    highlight-color: blue.lighten(90%),
    rows: 0, // Fig at the bottom with
    cols: 0, // no grid lines
  )
```

Code 8: Code to generate Figure 8

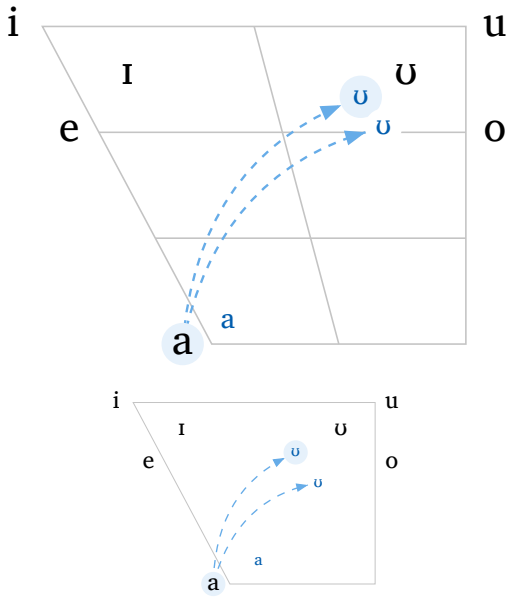


Figure 8: Flexible arrows and shifted vowels

2.2.3. 3D vowel space

As of version `0.6.0`, the function `#vowels-3d()` adds a third dimension to the trapezoid: rounding. It is based on the vowel space proposed by Ladefoged & Maddieson (1996, p. 283), where the familiar trapezoid is extruded into a prism. The front face of the prism is unrounded, and the back face is rounded. Front vowels sit on the left face of the prism, back vowels sit on the right face, and their depth reflects their degree of rounding: /i/ is the least rounded vowel and /u/ the most rounded one, while the low vowels /a/ and /ɒ/ meet halfway. Schwa is the only vowel inside the prism, halfway between the two faces. Edges hidden from the current viewpoint are dashed, as in Figure 9.

The function supports nine vowels: /i e ɛ a u o ɔ ɒ ə/. By default, all nine are plotted, so `#vowels-3d()` needs no arguments. A subset of them can be passed as the first argument using the same tipa-style input as `#ipa()`, for example `#vowels-3d("ieau")`. Unlike `#vowels()`, the function does not accept language names: any other vowel, or a name such as `"english"`, produces an error listing the nine available vowels.

The view is controlled by two angles. `rotation` spins the prism around its vertical axis: `0deg` shows it straight-on, positive values send the rounded face to the left, and negative values send it to the right. The default (`30deg`) reproduces the view in Ladefoged & Maddieson (1996). `tilt` sets how far above the prism we are looking from (default: `20deg`); negative values look at it from below. Both arguments accept either an angle or a plain number of degrees, so `rotation: 30` is the same as `rotation: 30deg`. As with `#vowels()`, `scale`, `width`, `height`, `rows`, and `cols` are also available.

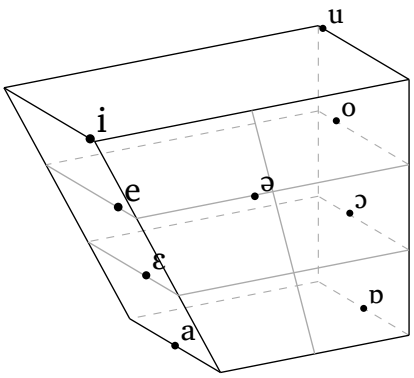


Figure 9: `#vowels-3d(scale: 0.6)`

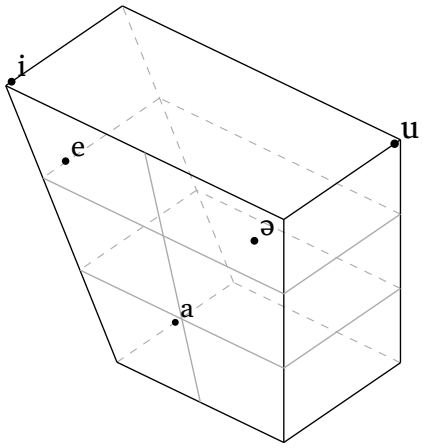


Figure 10: `#vowels-3d(.., tilt: 35deg, ..)`

To reinforce the 3D effect, vowels farther from the viewer are drawn smaller: in angled views such as the default, a vowel at the very back of the prism is drawn at 70% of its size, and vowels in between shrink gradually. Distance is recomputed for every combination of `rotation` and `tilt`, so the same vowel can be large in one view and small in another. In Figure 12, for example, the prism is rotated by `60deg`, which brings the unrounded front vowels close to the viewer and pushes the rounded back vowels away: compare the size of /e/ and /o/ in the two figures. Only vowel symbols and their dots are affected: lines keep their length and thickness, and nearer vowels are drawn on top of farther ones when they overlap. The effect fades out automatically as the view approaches a flat front (or back) view, where the prism looks like a regular trapezoid and differences in size would have no visible depth to explain them: `#vowels-3d(rotation: 0, tilt: 0)`, for instance, shows all vowels at the same size. Views from the top, the bottom, or the side keep the effect, since there the size of each vowel conveys the dimension that is hidden (height or front/back position). To turn the effect off in any view, use `depth-cue: false`.

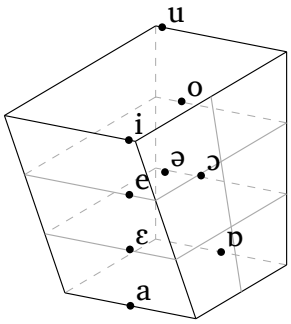


Figure 11: `rotation: 60deg, depth-cue: false`

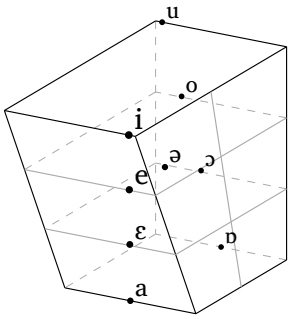


Figure 12: `rotation: 60deg, depth-cue: true`

Because `rotation` is an ordinary argument, a rotating prism can be shown in a presentation by generating one slide per angle. The loop in Code 9 creates 36 pages, one every 10 degrees, starting at the default view and completing a full turn. Placing the figure inside a box of fixed size keeps it centered as its outline changes from one angle to the next.

```
#for r in range(30, 390, step: 10) {
  page(align(center + horizon, box(
    width: 12cm, height: 7cm,
    align(center + horizon, vowels-3d(rotation: r)),
  )))
}
```

Code 9: Rotating `#vowels-3d()` across several pages

3. Phonetics

3.1. Sound shifts

The function `#sound-shift()` is meant for cases where a vowel trapezoid is no longer the best fit. This is often the case when we want to draw a schematic chain shift, merger, or split in which symbols must be placed freely in two-dimensional space. In `#sound-shift()`, each node is defined by a label and a position, and arrows connect nodes by referring to those labels directly. Optional arguments make it possible to curve arrows, circle nodes, and scale the whole representation up or down. Figure 13 shows a simple schematic representation of the Northern Cities Shift (Labov et al., 2008).

```
#sound-shift(
  nodes: (
    (label: "I", at: (-4.2, 2.8)),
    (label: "E", at: (-1.9, 1.0)),
    (label: "2", at: (0.9, 1.0)),
    (label: "O", at: (3.8, 1.0)),
    (label: "A", at: (2.4, -1.5)),
    (label: "\\ae", at: (-1.0, -1.5)),
  ),
  arrows: (
    ("E", "2"),
    ("2", "O"),
    ("O", "A"),
    ("A", "\\ae"),
    ("I", "E"),
    (from: "\\ae", to: "I", curved: true,
curve: 0.28),
  ),
  scale: 0.7,
  highlights: ("A",),
)
```

Code 10: Code to generate Figure 13

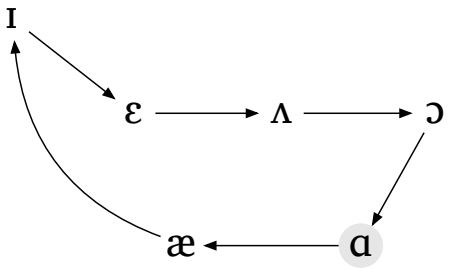


Figure 13: Schematic sound shift

3.2. Voice Onset Time

The function `#vot()` creates schematic timelines to illustrate negative, zero, and positive voice onset time values. This can be useful in introductory phonetics and phonology courses when discussing prevoicing, release, aspiration, and the onset of voicing. As shown in Code 11 and Figure 14, a minimal diagram for a negative VOT of −70 ms can be generated with `#vot(-70)`, but the function also supports localized labels, optional segmental annotation below each interval, and a range of aesthetic adjustments such as fill color and the display of schematic voicing waves. The resulting diagrams are intended as pedagogical illustrations rather than as direct plots from acoustic measurements, of course. Finally, as with other functions in **phonokit**, you can scale it up or down with the `scale` argument.

```
// minimalist function
#vot(-70)

// support for French and Portuguese labels
#vot(0, ui-lang: "pt")

// annotate diagram using IPA symbols
#vot(60, closure-segment: "t", interval-segment: "\\h", vowel-segment: "\\ae")

// various arguments to customize diagrams
#vot(
 25,
 ui-lang: "fr",
 voicing: false, // optionally hide illustrative waves
 fill-closure: blue.lighten(50%).desaturate(30%),
 fill-vowel: yellow.lighten(10%),

)
```

Code 11: Code to generate the four VOT diagrams in Figure 14

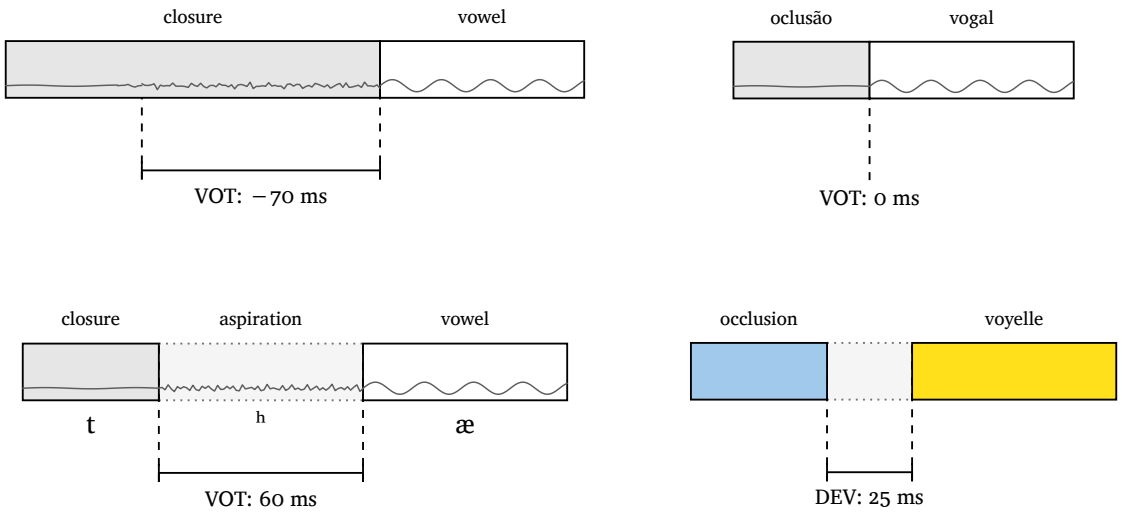


Figure 14: Examples of the `#vot()` function

3.3. Vowel dispersion

The function `#formants()` creates a vowel plot to illustrate vowel dispersion. The function makes use of the great Lilaq package (Mc-Zen, 2025). This can be handy in introductory phonology/phonetics courses to demonstrate how vowels overlap depending on how variable their F1 and F2 formant values are. The function allows for a number of arguments, including the standard deviation used for synthetic jitter (`sd` for F1, and `sd2` for F2 when specified; otherwise both use the same value) and the number of samples (defaults to 10 per vowel). Vowel labels represent mean F1 and F2 values. As with `#vowels()`, preset languages are also available, as shown in Figure 15. These preset values are schematic and intended only for pedagogical illustration. Several arguments are also available for aesthetic adjustments (point sizes, color, transparency, optional ellipses, etc.). You can also scale it up or down using the `scale` argument. Some of the available arguments are shown in Code 12, but the user should explore the function to experiment with the remaining arguments.

```
#formants("italian", scale: 0.6, ellipse: true, axis-size: 1.3em),
#formants("english", point-size: 100,
  point-color: luma(150),
  axis-size: 1.3em,
  vowel-size: 2.8em,
  grid: false,
  scale: 0.6
)
```

Code 12: Code to generate the two plots in Figure 15

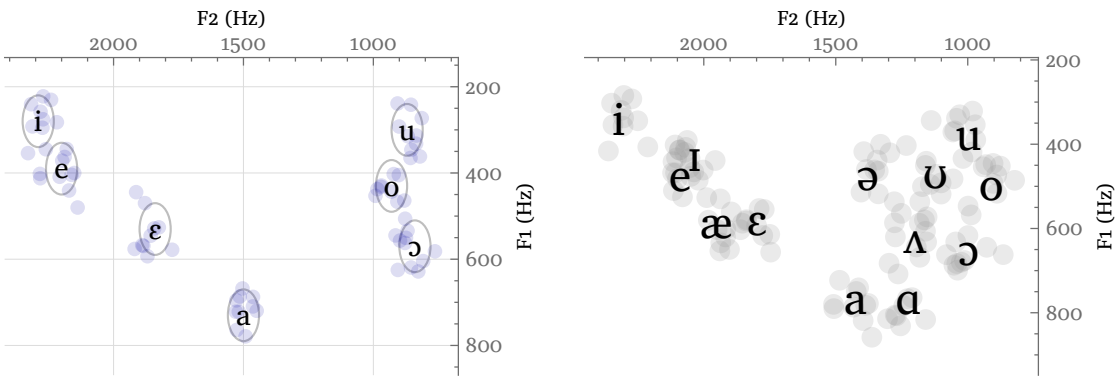


Figure 15: Vowel dispersion illustration

Note that the F1-F2 values used in `#formants()` are illustrative, and only serve as a pedagogical tool. The user may want to use the `seed` argument (default: `1`) to ensure that multiple figures use identical F1-F2 values to reproduce the same vowel positions.

If you would like to plot actual vowels from a real file, `#formants()` can also be used. The `source` argument can read a `csv` file and plot the vowels in it, as long as your file contains a header with `vowel`, `f1`, and `f2` columns. It's OK if the file has other columns, but these three are necessary.[2] Code 13 shows an example of the expected `csv` format.

[2] In `csv` mode, vowel means are computed directly from the observed F1 and F2 values for each vowel category, and ellipses use sample standard deviations.

```
vowel,f1,f2,speaker
i,342,2322,s1
i,360,2260,s2
i,334,2350,s3
i,351,2295,s4
i,347,2310,s5
e,476,2089,s1
e,500,2040,s2
```

Code 13: A sample `csv` file with vowel formants

Once you have your `csv` file, you can pass it to `#formants()` via the `csv()` function. Here, the file `formants_sample.csv` sits inside the `extras/` directory. The path to the file goes inside `csv()`. This is shown in Code 14.

```
#formants(source: csv("extras/formants_sample.csv"), scale: 0.8)
```

Code 14: Code to generate Figure 16

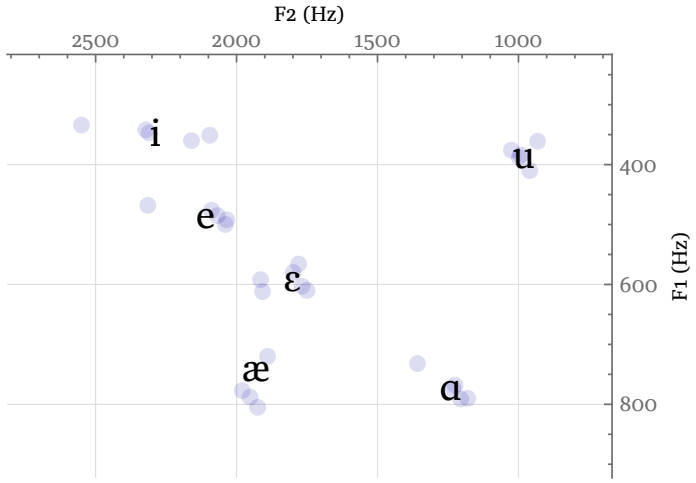


Figure 16: Vowels plotted from a `csv` file

4. SPE

Rewrite rules can be very complex, and an excellent package already exists to deal with their complexity in Typst (Breit, 2025, `linphon`). The problem is that, like autosegmental representations, too many degrees of freedom exist in SPE-like representations, not to mention the variation across scholars when it comes to symbols, brackets, etc. On the plus side, you can do a lot simply by employing primitives that already exist (e.g., matrices and arrows), so SPE rules are not as challenging to typeset as some other non-linear structures in phonology (at least simpler rules…). For that reason, **phonokit** only has two primitive functions to help create feature matrices, which in turn can be combined to form SPE-style rules (Chomsky & Halle, 1968).

```
#feat-matrix("p") #feat-matrix("\\ae") #feat-matrix("\\t tS") #feat-matrix("i")
```

Code 15: Generating matrices for the phonemes in "patchy", shown in Figure 17

The first function is `#feat-matrix()`, shown in Code 15. It outputs the maximal feature matrix for a given phoneme (with the option for 0 values if `all: true`). This can be useful in introductory courses, where students are introduced to the notion of distinctive features. The function is not able to compute *minimal* matrices, i.e., it can't figure out that feature A entails feature B given inventory I,[3] but it can be used in sequence to represent matrices for a given word, for example — see Figure 17. The user is free to adjust the inventory of features and their values, since there's variation in the literature. The function is based on the features in Hayes (2009).

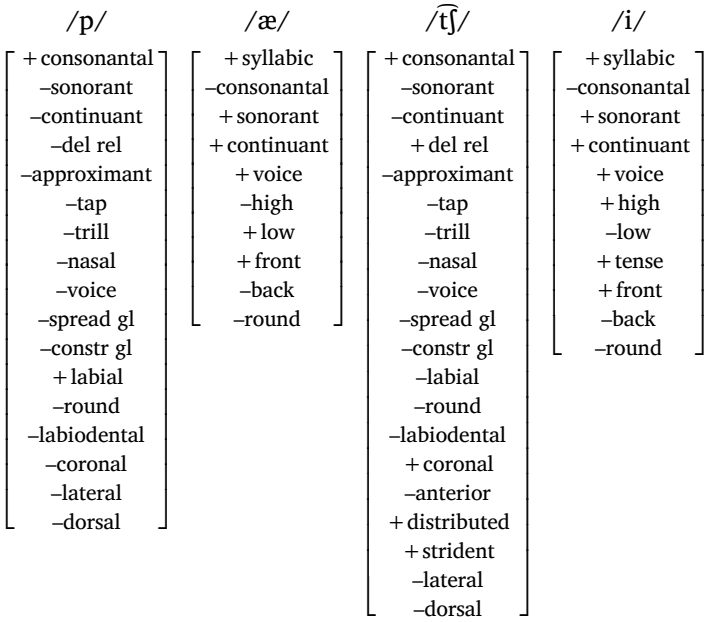


Figure 17: Matrices for the phonemes in the word "patchy"

Next, the function `#feat()` creates a matrix given a set of features. This is the function used in a rewrite rule, for example. The assimilation rule in Figure 18 is achieved with the code shown in Code 16 — notice that `alpha` notation requires a specific syntax, i.e., `X + "feat"` or `X + [#smallcaps("feat")]` if you prefer to use small caps. A helper function, `#blank()`, adds a long underline for the context of application in the rule. Likewise, `#a-r` adds an arrow using New Computer Modern (other arrows are available, such as `#a-l` ←, `#a-u` ↑, `#a-d` ↓, `#a-lr` ↔, `#a-sr` ⇝); see Section H.1.

```
#feat("+son", "-approx") #a-r
  #feat(sym.alpha + [#smallcaps("place")]) / #blank()\]#sub[#sym.sigma]
  #feat("-son", "-cont", "-del rel", sym.alpha + [#smallcaps("place")])
```

Code 16: Nasal place assimilation using `#feat()`

[3] But see the Fonology package for R (Garcia, 2026).

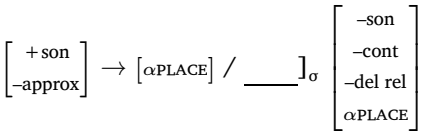


Figure 18: A nasal place assimilation rule

5. Prosody module

5.1. Sonority

When discussing the sonority principle in introductory phonology courses, it is often useful to illustrate relative sonority with a visual representation. The function `#sonority()`, based on the Fonology package for R (Garcia, 2026), plots phonemes and their relative sonority profiles. The function is based on the sonority scale in Parker (2011, p. 18). You may want to customize the sonority scale in question to your needs and preferences. In that case, download or clone the package and adjust the scale as needed in the source code (`sonority.typ`).

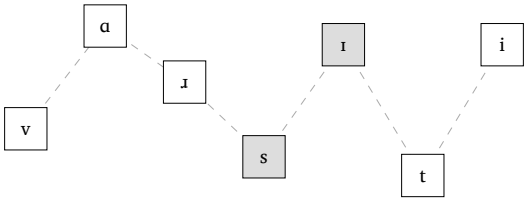


Figure 19: `#sonority("vA \\*r .sI.ti", scale: 0.7)`

Figure 19 shows an example for the word “varsity”. If syllable boundaries are detected in the input, the function alternates between white and gray fills to distinguish each syllable. If no boundaries are detected, all boxes will be white by default.

5.2. Syllables

We start with an essential representation, namely the syllable. Two options are available: `#syllable()` for a classic onset-rhyme representation (Figure 20), and `#mora(.., coda: true)` for a moraic representation (Figure 21). The latter option allows you to define whether or not codas project a mora (`coda: true`). These two functions are used for single-syllable representations only. As can be seen in the figures, these functions take as input a string that should be familiar given the discussion about `#ipa()` in Section 2.1.



Figure 20: `#syllable("\\t tS \\ae t")`

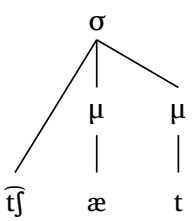


Figure 21: `#mora("\\t tS \\ae t", coda: true)`

Vowel length is also represented in both `#syllable()` and `#mora()`, as can be seen in Figure 22 and Figure 23, respectively. The crucial element here is the use of `:`, which triggers the : symbol for both representations. In the moraic representation, two moras branch out of the vowel, as expected.

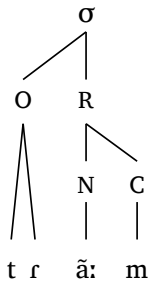


Figure 22: `#syllable("tR \\~ a:m")`

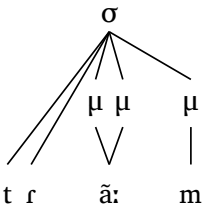


Figure 23: `#mora("tR \\~ a:m", coda: true)`

The dimensions of each representation adjust as a function of how many segments are found in the input. As such, more complex onsets, nuclei or codas result in wider representations that respect a safe and consistent between-segment distance. Figure 24 illustrates this with an extreme example.

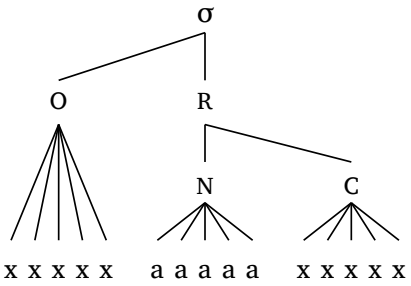


Figure 24: How spacing is managed

We often need to adjust the size of a representation as a whole. But doing so can be problematic if the text and the representation itself behave independently. Here, however, representations can be easily adjusted with the argument `scale`, which takes care of both line width and text size. Examples are shown in Figure 25, Figure 26 and Figure 27.



Figure 25: Scale 0.75



Figure 26: Scale 0.5



Figure 27: Scale 0.25

5.3. Feet

Next, we examine metrical feet (Figure 28 and Figure 29). These functions are designed to deal with a single foot where all syllables are footed by definition, since unfooted syllables have nowhere to attach to (see Section 5.4). A period `.` is used in the code to indicate syllabification and a single apostrophe

`'` is used to indicate which syllable is the head of the foot. This allows us to easily create trochees and iambs. Naturally, you are free to generate non-binary feet, as the function can handle them as well (dactyls in Figure 30 and Figure 31).

Can I choose my own symbols? *Yes*. If you prefer "Ft" instead of Σ, for example, see Section B.

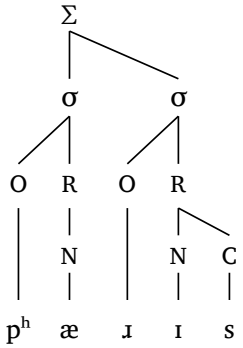


Figure 28: `#foot("'p \\h \\ae.\\*r Is")`

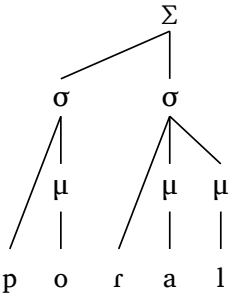


Figure 29: `#foot-mora("po.'Ral", coda: true)`

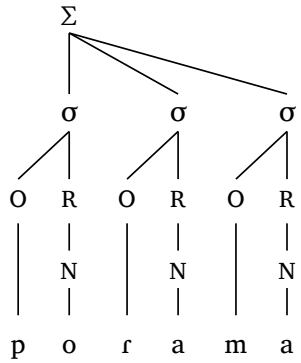


Figure 30: `#foot("'po.Ra.ma")`

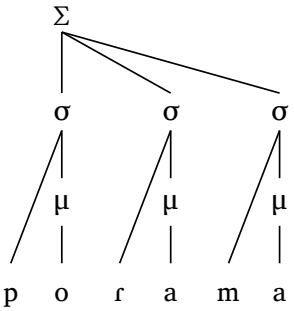


Figure 31: `#foot-mora("'po.Ra.ma")`

Geminates are also represented by the functions `#foot()` and `#foot-mora()`. In onset-rhyme representations, a geminate will be linked to the coda and the following onset, as expected. In moraic representations, the user will probably want to define `coda: true` to represent geminates in a traditional fashion. Figure 32 and Figure 33 show a disyllabic word containing a geminate in both onset-rhyme and moraic representations.

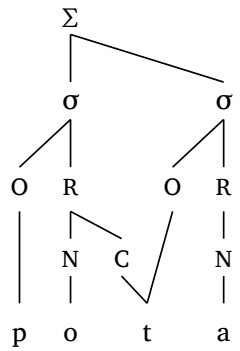


Figure 32: `#foot("'pot.ta")`

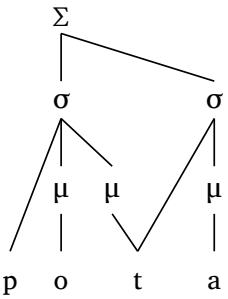


Figure 33: `#foot-mora("'pot.ta", coda: true)`

Extreme cases are important to test how adaptable the function is when it comes to line crossings, a key problem in prosodic representations. When the head of a domain (the foot here) is at an edge of a long string, it is challenging to avoid crossing or overlapping lines. As can be seen in Figure 34, the height of Σ is proportional to the width of the representation to avoid superposition of lines (this is the default behavior, `spacing: "auto"`, discussed below).

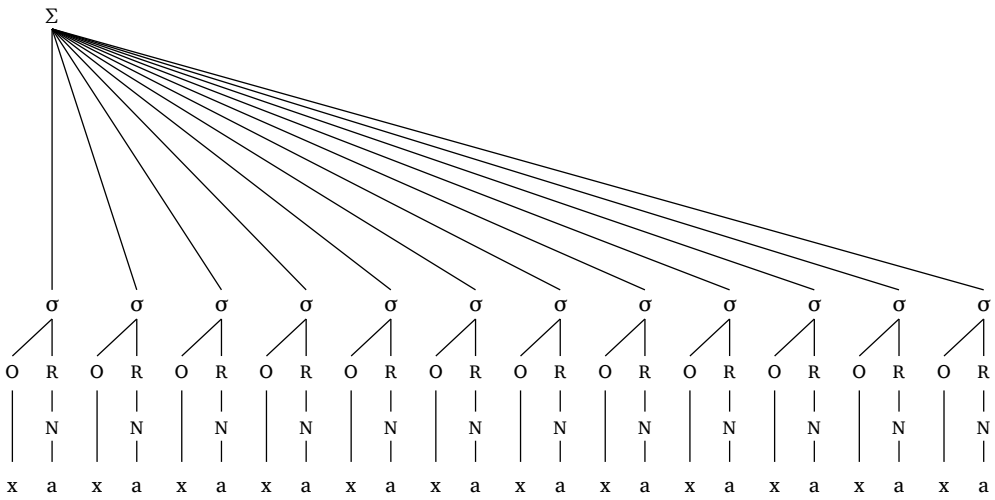


Figure 34: An extreme case

5.4. Prosodic words

Finally, we arrive at prosodic words (PWd), which bring together syllables and feet. This is where the user has more options, given the metrical parameters involved. Parentheses `()` are used to define feet, which means that any syllable *outside* the foot will be linked directly to the PWd. Next, an apostrophe `'` symbolizing stress (both primary *and* secondary) is used to indicate the head of each foot. Finally, the argument `foot: "R"` or `foot: "L"` is used to determine which foot in the PWd contains the primary stress in the word (in cases where more than one foot is present in a given PWd).

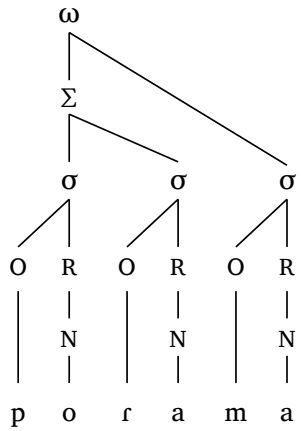


Figure 35: `#word("('po.Ra).ma")`

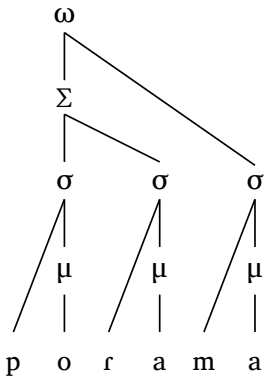


Figure 36: `#word-mora("('po.Ra).ma")`

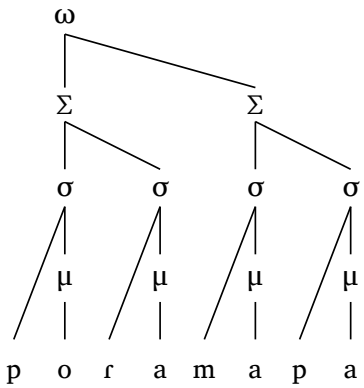


Figure 37: When `foot: "L"`

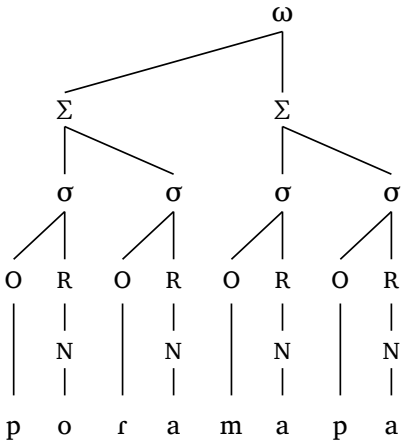


Figure 38: When `foot: "R"` (default)

It is worth noting that *all* lines are straight in the prosody module (this is by design), so curved lines are not a possibility. Consequently, in extreme scenarios, e.g., where an unfooted syllable is *very* far away from the head foot of a given PWd, the height of the representation will be adjusted accordingly to avoid line crossings. This will inevitably create taller figures, as already mentioned. Figure 39 presents a hypothetical scenario to demonstrate how the function deals with extreme cases.

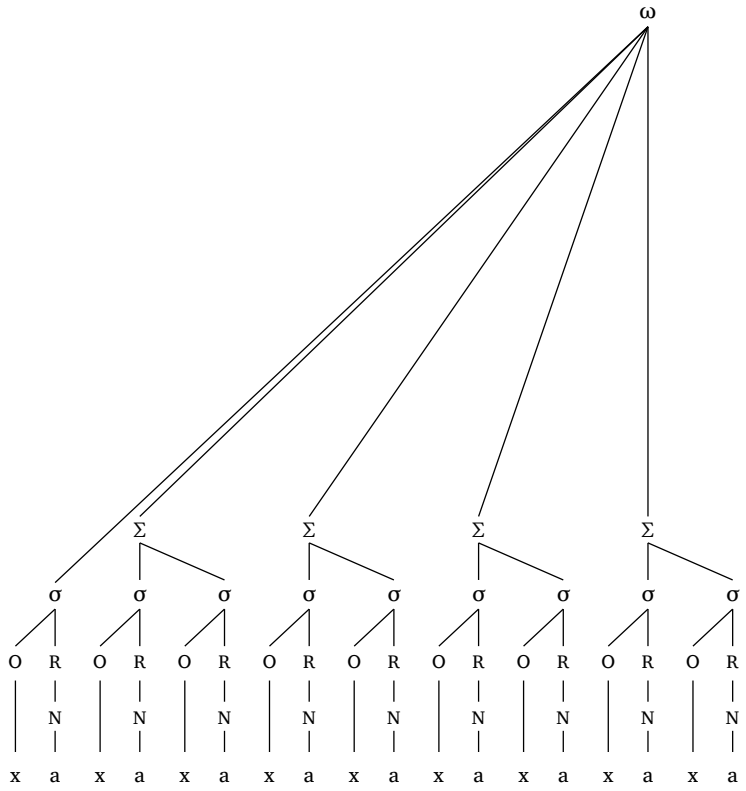


Figure 39: Unfooted syllable far away from head foot

As of version 0.5.3, you can also adjust the vertical spacing between prosodic levels in the functions discussed above. While each function is designed to optimize vertical spacing, you might have your own preferences, so the argument `distance` allows for some adjustments in the form of an array where you specify the level you wish to target and the change in the spacing. Levels begin from 0 and from the top, so when using `#word()`, level 0 targets the prosodic word, level 1 targets the foot (if it's present), and so on. The distance is then relative to the defaults, such that `distance: ((0, 1.2),)` would increase the distance of the prosodic word level by 20%, and `distance: ((0, 0.8),)` would reduce it by 20%. What is scaled is the visible line between two levels, and that line never gets shorter than a small minimum, so you can't make a line disappear (negative numbers aren't allowed, either).

As of version 0.5.20, vertical spacing adapts to each representation by default (`spacing: "auto"`): a line never gets shorter than the line directly below it, wider branches get more height so they don't flatten out, and levels that aren't present (e.g., feet in `#word("trist")`) take up no space. If you'd rather have constant spacing, for instance to align two representations side by side, use `spacing: "fixed"`. In both cases, `distance` is applied on top.

```
#word(
  "(pa.ra)(pa.ra)",
  distance: (
    (0, 0.7), // PWd
    (1, 0.9), // Ft
    (2, 0.7), // Syl
    (3, 1),   // OR
    (4, 1),   // N
  ),
)
```

Code 17: Code to generate Figure 40

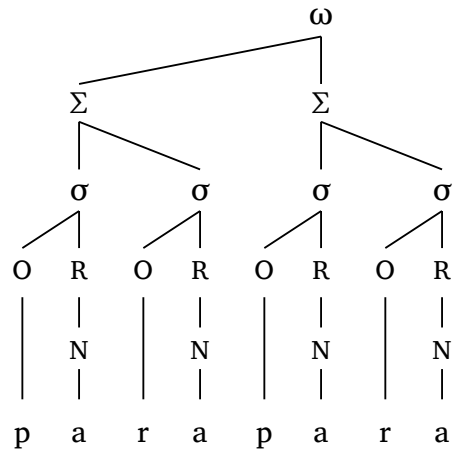


Figure 40: Custom spacing between levels

5.5. Metrical grid

The function `#met-grid()` allows you to create a metrical grid using × to indicate prominence (e.g., Zsiga (2024, p. 373)). As with all functions in the package, the goal here is to create high-quality output with *minimal effort*, that is to say, with functions that are as intuitive and parsimonious as possible. Figure 41 and Figure 42 show grids for the word *butterfly*.

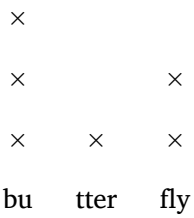


Figure 41: Input as string

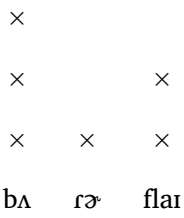


Figure 42: Input as tuple (IPA-compatible)

```
#met-grid("bu3.tter1.fly2") // Input as string
#met-grid(("b2", 3), ("R \\schwar", 1), ("flaI", 2)) // Input as tuple
```

Code 18: Metrical grids shown in Figure 41 and Figure 42

6. Autosegmental phonology

Another key function in **phono*kit*** is `#autoseg()`, introduced in version 0.3.5. The function allows you to represent either features or tones on a separate tier. More importantly, the function is able to show linking and delinking, floating tones, as well as contour tones. Figure 43 demonstrates how `#autoseg()` works in a simple scenario of feature spreading.

```
#autoseg(
    ("k", "\\ae", "n", "t"),
    features: ("", "", "[+nas]", ""),
    links: ((2, 1),),
    spacing: 1.0,
    arrow: false,
    dash: "dashed", // default
  )
```

Code 19: Code to generate Figure 43

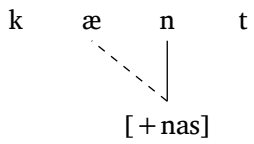


Figure 43: Feature spreading

As can be seen in the figure, `#autoseg()` accepts the same inputs as `#ipa()`. Here, inputs are arrays, so each phoneme must be entered individually. This results in more flexibility, as you can add empty spaces, symbols for domain boundaries, etc. The argument `features` works the same way. In this particular case, only one feature is present (in position 2, since we count from zero). That position is aligned with the segment "n": this automatically draws a vertical line linking segment to feature. Indices are key because the argument `links` depends on them. For the spreading to be represented here, `links` is defined as **an array of tuples** `((2,1),)`, hence the commas. Read this as "draw a link from index 2 to index 1". Finally, `spacing` allows you to conveniently set the inter-segmental spacing (lines and links are adjusted accordingly). The argument `arrow` simply asks whether an arrow head should be added to the linking line (`true` or `false`).

Multiple links are easy to implement. Figure 44 shows an example of metaphony in Brazilian Veneto (Garcia & Guzzo, 2023) where two vowels are targeted by the spreading of [+high]. The source of the process is position 6 ("i") and the targets are positions 3 ("e") and 1 ("o"), hence the array `((6,3), (6,1))` in the code. Finally, notice that the argument `gloss` allows for quick annotation.

```
#autoseg(
    ("z", "o", "v", "e", "n", "-", "i"),
    features: ("", "", "", "", "", "", "[+hi]"),
    links: ((6, 3), (6, 1)),
    spacing: 0.5, // better spacing
    arrow: true,
    gloss: [_young_.#smallcaps("pl")],
  )
```

Code 20: Code to generate Figure 44

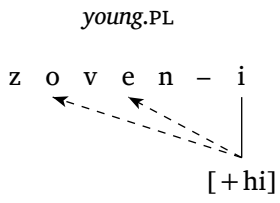


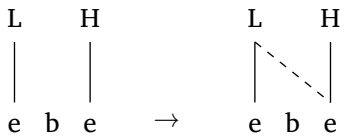
Figure 44: Metaphony in Brazilian Veneto

6.1. Tones

One of the arguments in `#autoseg()` is `tone` (`true` or `false`). This allows us to "flip" the representation vertically to show tones above the segmental tier. Take a look at Figure 45, which shows a case of low tone spreading without delinking in Nupe (Zsiga, 2024). We can easily add multiple instances of the function to represent processes with relatively concise code.

```
#autoseg(
  ("e", "b", "e"),
  features: ("L", "", "H"),
  spacing: 0.5,
  tone: true,
  gloss: [],
)
#a-r // arrow
#autoseg(
  ("e", "b", "e"),
  features: ("L", "", "H"),
  links: ((0, 2),),
  spacing: 0.5,
  tone: true,
  gloss: [èbě _pumpkin_],
)
```

Code 21: Code to generate Figure 45



èbě *pumpkin*

Figure 45: Tone spreading

Floating tones can be added with the `float` argument. Code 22 demonstrates how to add such a tone and how to draw a circle around it with the `highlight` argument, which can be used with any tone, of course.

```
#autoseg(
      ("i", "@", "N", "k", "a"),
      features: ("H", "", "L", "", "H"),
      highlight: (2,),
      spacing: 1.0,
      tone: true,
      float: (2,),
    )
```

Code 22: Floating tone

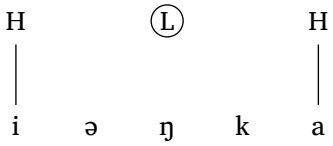


Figure 46: Floating tone

Spreading from a contour tone involves a more complex scenario. In Figure 47, the vowel in position 2 is linked to two tones. To create this representation, the element in position 2 inside `features` must be itself a tuple with the two tones in question. This will already add the solid lines connecting the vowel to the two tones. The `links` argument then takes care of the dashed line: `(((2, 0), 1),)`. This merely says "look at position 2 and pick the first element there, i.e., `(2,0)`, then draw a dashed line from that element to position 1". Tuples inside tuples are essential here.

```
#autoseg(
  ("m", "1", "A", "u"),
  features: ("", "", ("H", "L"), ""),
  links: (((2, 0), 1),),
  tone: true,
  highlight: ((2, 0),),
  spacing: 1.0,
  arrow: true,
),
```

Code 23: Contour tone

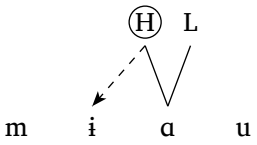


Figure 47: Contour tone

Finally, let's take a look at a three-step process (Zsiga, 2024). In this example, we have three instances of `#autoseg()` (one for each stage in the process). Here, we have delinking and segments that share a tone, which require some branching. Delinking is an essential process to represent, and that's what the argument `delinks` does. If you examine Code 24, you will notice that the third function specifies `delinks: ((3, 3),)`. This merely states that the link created between position 3 and itself must be delinked. Both `links` and `delinks` work as arrays of tuples.

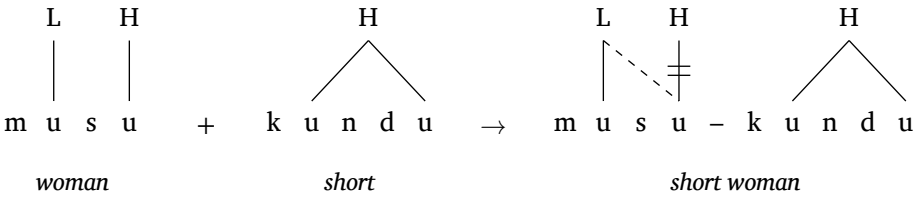


Figure 48: OCP effects in Vai (Zsiga, 2024)

To create a tone that is underlyingly linked to more than one segment, we use the `multilinks` argument. Unsurprisingly, this will require tuples inside tuples once again. For example, the result of the process in question is represented by the third `#autoseg()` in Code 24. There, we see `multilinks: ((6, (6, 9)),)`, which means "search for position 6 (this is the 'u' in 'kun'), then link that to positions 6 and 9 *at the same time*". The function automatically centers the tone between the two segments in question (it will center the tone regardless of how many segments it is linked to), and it also "erases" the vertical line inserted by the `features` argument.

```
#autoseg(
      ("m", "u", "s", "u"),
      features: ("", "L", "", "H"),
      tone: true,
      spacing: 0.5, // keep consistent
      baseline: 37%,
      gloss: [_woman_],
    ) +
    #autoseg(
      ("k", "u", "n", "d", "u"),
      features: ("", "H", "", "", ""), // H at position 1, but will be repositioned
      tone: true,
      float: (1,), // Mark H as floating so it doesn't draw vertical stem
      multilinks: ((1, (1, 4)),), // H links to segments at positions 1 and 4
      spacing: 0.5,
      baseline: 37%,
      arrow: false,
      gloss: [_short_],
    ) #a-r
    #autoseg(
      ("m", "u", "s", "u", "-", "k", "u", "n", "d", "u"),
      features: ("", "L", "", "H", "", "", "H", "", "", ""),
      links: ((1, 3),), // Link between L and H
      delinks: ((3, 3),), // Delink line between position 3 and itself
      arrow: false,
      multilinks: ((6, (6, 9)),), // Automatically removes vertical line in position 6
      tone: true,
      baseline: 37%, // Adjust as needed to customize vertical position
      spacing: 0.50,
      gloss: [_short woman_],
    )
```

Code 24: Code to generate Figure 48

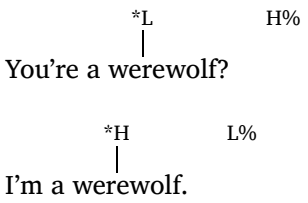
Autosegments are difficult to typeset because there are many degrees of freedom involved, which means functions start to become too convoluted for the benefit they provide (see Section 6.3). There is probably a sweet spot, a point beyond which a function of this type becomes impractical because there are simply too many arguments. `#autoseg()` attempts to be intuitive while covering spreading, delinking, one-to-many and many-to-one relationships, floating tones, and highlights with circles. I believe these cover the vast majority of scenarios with precision and symmetry. The function also provides convenient arguments for horizontal (`spacing`) and vertical (`baseline`) positioning, which can be handy in processes where you may want to adjust the position of the representation relative to an arrow or any other symbols you may want to add outside the function *per se*.

6.2. Prosody with ToBI

Adding ToBI labels to strings can be achieved with the function `#int()`. One of its main arguments is `line`, which allows the user to "turn off" the vertical line (e.g., for boundary tones). The examples below are from Zsiga (2024). By default, the font size of tones is set to `0.8em`.

```
You're a we#int("*L")rewolf?#h(2em)#int("H%", line: false)


I'm a wer#int("*H")ewolf.#h(2em)#int("L%", line: false)
```



Code 25: ToBI labels

In the vast majority of cases, the strings in Code 25 will be in numbered examples that can be cross-referenced in the text. To see how `#int()` can work in those scenarios, see Section 10. Note that `#int()` is not meant to be used in numbered/unnumbered lists: you should always favour numbered examples instead.

6.3. Multi-tier representations

Thus far, the functions we've seen are single-purpose: `#syllable()` only creates syllables. The advantage of such an approach is that the function can offer minimal syntax, which makes it easy to use and very user-friendly. The disadvantage is that the user can only produce one type of output. Certain phonological structures, however, have too many degrees of freedom for a single-purpose function to be enough. This is why the function `#multi-tier()` exists: it gives you the freedom to create a wide range of non-linear structures. It is much more flexible, but that comes with more complicated syntax and more arguments, by definition.

Let us look at three use cases. First, let's reproduce a figure from Booij (2012, p. 159) on the interface of phonology and morphology. This is a good scenario to test the function because the figure requires multiple tiers. The figure is shown in Figure 49, and the code needed is shown next to Figure 50, which also displays the grid to help you see the position of each element. The main argument is `levels`, which should be relatively familiar given the discussion on `#autoseg()` earlier. Think of the function as a big grid where you position elements relative to rows (i.e., levels) and columns.

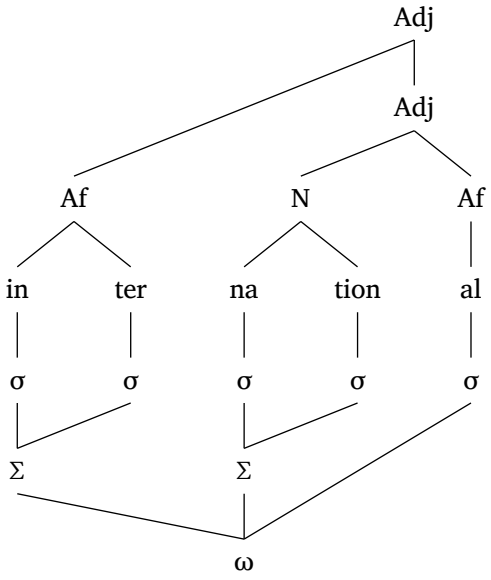


Figure 49: Morphology and phonology

```
#multi-tier(
  show-grid: true, // ← To help you see the grid
  levels: (
    ("", "", "", "", ("Adj", 3.5)),
    ("", "", "", "", ("Adj", 3.5)),
    ("", ("Af", 0.5), "", ("N", 2.5), "Af"),
    ("in", "ter", "na", "tion", "al"),
    ("sigma", "sigma", "sigma", "sigma", "sigma"),
    ("Sigma", "", "Sigma", "", ""),
    ("", "", "omega", "", ""),
  ),
  links: (
    ((0, 4), (2, 1)), // Adj → Af
    ((1, 4), (2, 3)), // Adj → N
    ((2, 1), (3, 0)), // Af → in
    ((2, 3), (3, 2)), // N → na
    ((5, 0), (4, 1)), // Ft → Syl
    ((5, 2), (4, 3)), // Ft → Syl
    ((6, 2), (5, 0)), // PWd → Ft
    ((6, 2), (4, 4)), // PWd → Ft
  ),
),
```

Code 26: Code to create Figure 50

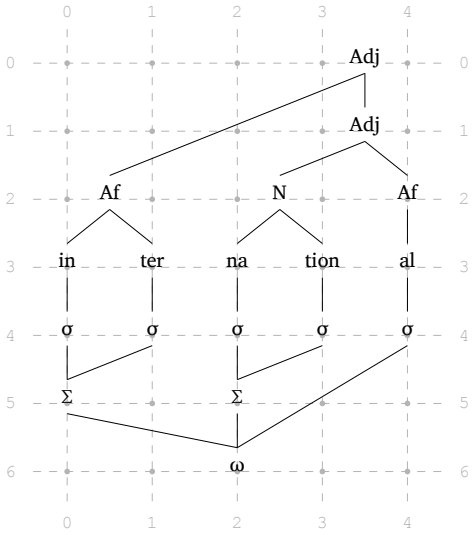


Figure 50: Morphology and phonology

You can think of `#multi-tier()` as a more powerful version of `#autoseg()`, and you will recognize many common elements between the two. For example, `float` is again an argument here, which allows you to have elements in the representation that are not linked to anything. This is important because `#multi-tier()` automatically links an element (with a single link) to the element on the next level (row). In Figure 50, you can comment out the contents of the `links` argument to see what the figure looks like in its bare form. Likewise, `highlight` can also be used if you need to circle a particular element in the representation.

Although the function still supports referencing nodes by their coordinates, as of version `0.5.7`, non-empty labels also receive automatic names in reading order (`adj1`, `adj2`, `sigma3`, etc.). This means that you can avoid coordinates altogether and simply refer to nodes by name. Figure 51 repeats Figure 50 for convenience. Notice that labels are **case-insensitive** in Code 27: both `sigma` and `Sigma` are assigned names of the form `sigmaX`, where `X` corresponds to the nth occurrence of `sigma` in the code. In practice, this usually makes node references easier to write and read. That said, coordinate references may still be preferable for levels containing IPA-heavy strings or other non-alphanumeric labels, since automatic normalization can make the generated node names less transparent and more dependent on reading order. For example, `/p@lis/` will be normalized to `p-lisX`, which isn't necessarily intuitive. If you want to inspect the exact final names generated by the function, use `show-refs: true`, which prints the usable reference below each eligible node: auto-generated names for string nodes and coordinate fallbacks for content nodes. Auto-labelling only applies to string nodes, not content nodes. In Typst, "this is a string" is a string, whereas [this is content] is content (the key being the square brackets and the absence of quotes). Content is more flexible and can include, for

example, math mode. Content nodes therefore do not receive automatic labels; if you need to refer to one, use coordinates instead.

```
#multi-tier(
  levels: (
    ("", "", "", "", ("Adj", 3.5)),
    ("", "", "", "", ("Adj", 3.5)),
    ("", ("Af", 0.5), "", ("N", 2.5), "Af"),
    ("in", "ter", "na", "tion", "al"),
    ("sigma", "sigma", "sigma", "sigma", "sigma"),
    ("Sigma", "", "Sigma", "", ""),
    ("", "", "omega", "", ""),
  ),
  links: (
    ("adj1", "af1"),
    ("adj2", "n1"),
    ("af1", "in1"),
    ("n1", "na1"),
    ("sigma2", "sigma6"),
    ("sigma4", "sigma7"),
    ("sigma5", "omega1"),
    ("sigma6", "omega1"),
  ),
),
```

Code 27: Code to create Figure 51

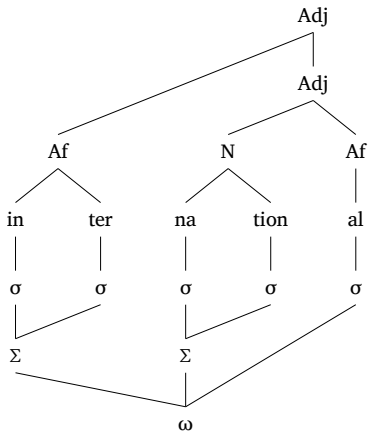


Figure 51: Morphology and phonology

By default, elements are automatically placed on the grid, but you may want to specify a different position. In Figure 50, for example, four elements are placed at fractional positions on the grid (both “Adj” at column 3.5, “Af” at 0.5, and “N” at 2.5). This is made possible by defining the position for the element in question as a tuple. Therefore, “N” is specified as `("N", 2.5)`, which places “N” in between “na” and “tion”, found in the level below. You can also add a third argument for the vertical position if you ever want to place an element in between levels (not shown here). The argument `levels` will also render numbers as subscripts automatically. Finally, you may have noticed that the function automatically detects Greek letters, so if you add “sigma”, it will render as σ.[4]

You can use `#multi-tier()` to create a wide range of representations that are not easily generated by single-purpose functions. Structures used in Government Phonology or CV Phonology, for instance, can be easily generated. Figure 52 is from Goad (2012, p. 355).

[4] **phonokit** also provides a list of the most common Greek letters used in phonology, which will always render without italics. See Section H.2.

```
#multi-tier(
  levels: (
    ("O", "R", "", "O", "R", "O", "R"),
    ("", "N1", "", "", "N2", "", "N3"),
    ("", "x", "x", "x", "x", "x", "x"),
    ("", "", "s", "t", "E", "m", ""),
  ),
  links: (
    ("r2", "x2"),
  ),
  ipa: (3,),
  arrows: (
    ("t1", "s1"),
    ("r2", "r1"),
  ),
  arrow-delinks: (
    (1,)
  ),
  spacing: 1,
)
```

Code 28: Code to create Figure 52

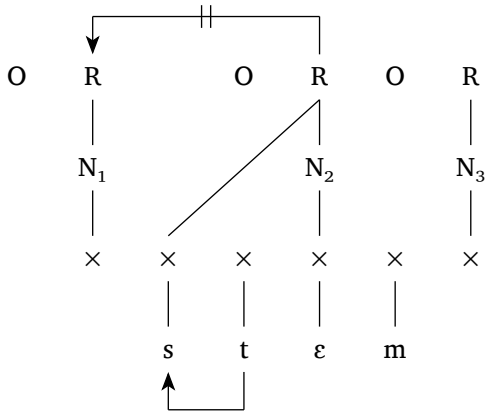


Figure 52: Government Phonology

As we examine Figure 52 and its code, some arguments are worth discussing in more detail. First, `ipa` allows you to specify which level contains IPA symbols that follow the conventions of **phono*kit*** (Section A). This makes it easy to add IPA symbols to the representation. Next, the argument `arrows` is responsible for the arrows you see in the figure. Arrows follow right-angle paths, and can be "cut off" with the `arrow-delinks` argument. Finally, `#multi-tier()`, like many other functions discussed here, also allows for sizing adjustments with arguments such as `scale`, `spacing`, `level-spacing`, `stroke-width` (which is used in Figure 53 below). As already mentioned, the flexibility offered by the function results in a much richer set of arguments, which in turn require a bit more time to get used to.

Finally, let's reproduce another figure, this time adapted from Carvalho (2017, p. 7). This representation also shows another argument, `tier-labels`, which is very practical if you ever want to label levels in your representation. Figure 53 also shows how to draw a dashed link between two elements, and how to `highlight` a particular element.

```
#multi-tier(
  levels: (
    ("T", "", "R", ""),
    ("O1", "n1", "O2", "n2"),
    ("x", "", ("x", 2.5), ""),
    ("o", "", ("N", 2.5), ""),
    ("", "", ("V", 2.5), ""),
  ),
  links: (
    ("n21", "x2"),
  ),
  dashed: (
    ("x1", "n1"),
  ),
  level-spacing: 1.2,
  highlight: (
    "o11",
  ),
  spacing: 1,
  stroke-width: 0.7pt,
  tier-labels: (
    (1, "C-plane"),
    (2, "skeleton"),
    (3, "V-plane"),
  ),
  scale: 1,
)
```

Code 29: Code to create Figure 53

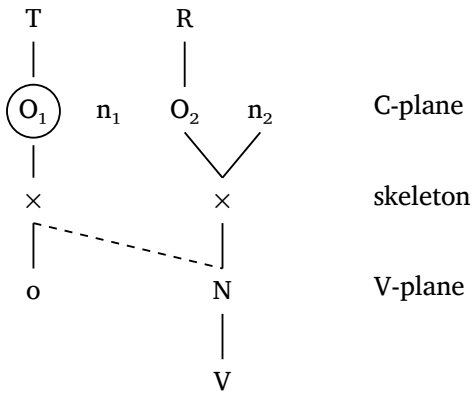


Figure 53: CV phonology

6.3.1. Help with labels

The argument `show-grid` can be helpful for understanding the layout of a representation, so you know exactly where nodes are located. Automatic labelling is also useful, since it results in more minimal code. However, sometimes you may have a lot of nodes, or nodes that involve IPA transcription, which means labels will be normalized differently. As of version `0.5.8`, a new argument makes `#multi-tier()` more efficient: `show-refs`, shown in Figure 54.

The argument in question provides a more targeted debugging aid. It prints the usable reference for each non-empty node directly on the figure, making it easier to add links without having to infer sources and targets. Since automatic labels are only generated for string nodes, content nodes in Figure 54 fall back to coordinates instead (which, of course, also work for string nodes). You can simply set `show-refs: true` while you complete your `links` and turn it off later.

```
#multi-tier(
    show-refs: true,
    levels: (
      ("", "", "", "/p@lis/", "", "", ""),
      ("", "", "", [#smallcaps("gen")\(erator)], "", "", ""),
      ("", ("[p@lis]", 1.5), ("[plis]", 2.5), ("[p \\r lis]", 3.5), ("[lis]", 4.5), ""),
      ("", [#smallcaps("con")\(straints) #h(1em)$arrow$], "",
[#smallcaps("eval")\(uation)], "", "", ""),
      ("", "", "", [* [output] *], "", "", ""),
    ),
    links: (
      ((1, 3), "p-lis2"),
      ((1, 3), "plis1"),
      ((1, 3), "lis1"),
      ("p-lis2", (3, 3)),
      ("plis1", (3, 3)),
      ("lis1", (3, 3)),
    ),
    float: (
      (3, 1),
    ),
    ipa: (0, 2),
  )
```

Code 30: Code to create Figure 54

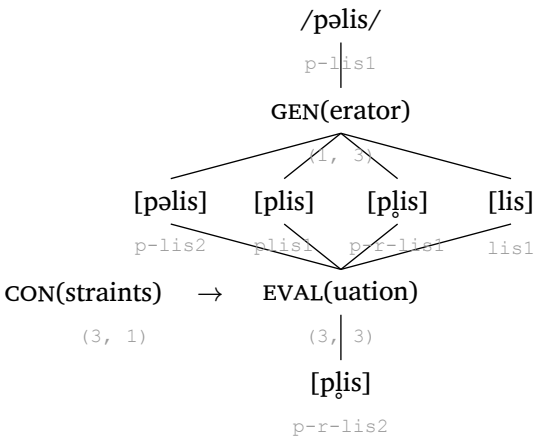


Figure 54: Illustration of /pəlis/ → [pl̥is] classical OT

7. Feature geometry

Feature geometry is a natural progression from functions that cover autosegmental phonology and multi-tier representations. It presents some important challenges, including the user interface: as already mentioned, if too many degrees of freedom are available, a function becomes too convoluted, which compromises its use. Below I describe two special functions that are dedicated to feature geometry:

`#geom()`, which takes care of any given representation, and `#geom-group()`, designed to represent multiple trees and processes.

7.1. Single trees

Let us begin by reproducing the full representation for consonants presented in Clements & Hume (1995, p. 292). If you inspect Code 31, you will notice that the arguments of the `#geom()` function are fairly straightforward: if you know what nodes you need to add, you can almost guess what the argument will be called. Note that if you add a node n, the function automatically builds any parent node $n-1$ needed. For example, by adding `voice: true`, `laryngeal` is added; and by adding `anterior: true`, `coronal` ([cor]) is also added, so the code in question didn't need to specify `coronal: true`.

```
#geom(
  root: ("±son", "±approx", "-vocoid"),
  spread: true,
  constricted: true,
  nasal: true,
  voice: true,
  labial: true,
  coronal: true, // ← implied
  anterior: true,
  distributed: true,
  dorsal: true,
  continuant: true,
)
```

Code 31: Code to create Figure 55

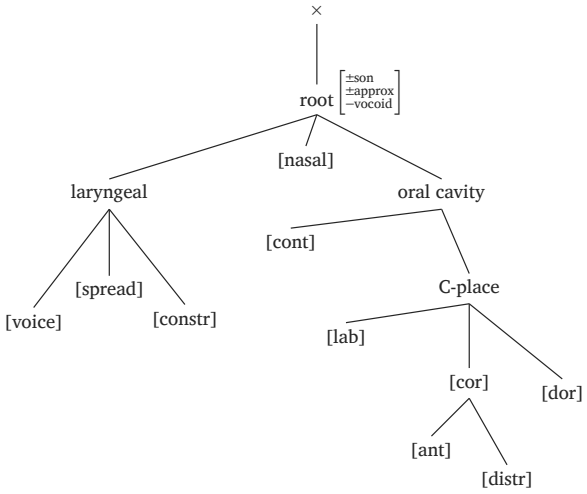


Figure 55: Consonants

```
#geom(
  root: ("+son", "+approx", "+vocoid"),
  spread: true,
  constricted: true,
  nasal: true,
  vplace: true, // ← implied
  labial: true,
  aperture: (true, true, true),
  coronal: true,
  anterior: "-",
  distributed: true,
  dorsal: true,
  continuant: true,
  scale: 0.9,
)
```

Code 32: Code to create Figure 56

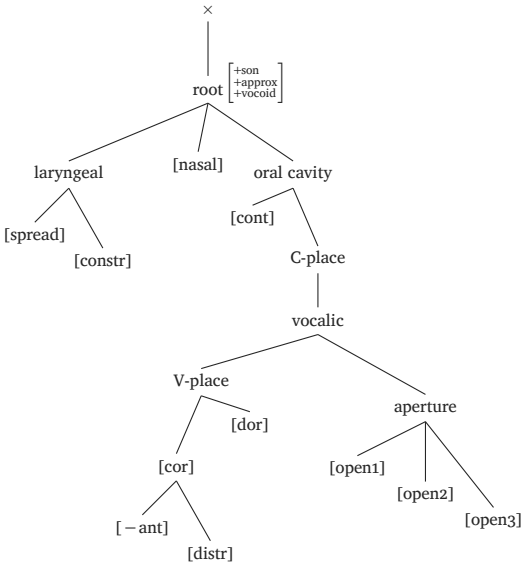


Figure 56: Vocoids

Arguments accept both boolean and string values. Thus, `nasal: true` simply adds [nasal], while `nasal: "+"` adds [+nasal]. In general, longer labels are abbreviated in the output, but arguments are never abbreviated, so the `anterior` argument generates [ant] in the actual representation.

Figure 56 reproduces the full representation for vocoids, again based on Clements & Hume (1995, p. 292). The argument `vplace: true` tells the function that place-related arguments should be attached to V-place, not C-place. Alternatively, the argument `vocalic: true` has the same effect, and so does `aperture` (so, technically, `vplace` is redundant in Code 32). Finally, notice the `scale` argument, which lets you easily adjust the overall size of your representation.

Thus far, we have seen that the arguments of `#geom()` are fairly straightforward: if you know what nodes you need, you know how to add them with the function. However, the goal of **phono*kit*** is to have minimal syntax and be as user-friendly as possible. For that reason, several phonemes come pre-built in `#geom()`: you simply need to use the `ph` argument and give it a phoneme (same conventions used in `#ipa()` discussed in Section 2.1).[5] You can see some examples in Figure 57 – Figure 60, where you will also note timing units added by default (×). You can disable timing units by adding `timing: false`, or you can add morae instead (`timing: mora` or `timing: mu`). The `timing` argument also takes an array for long vowels: `timing: ("x", "x")`.

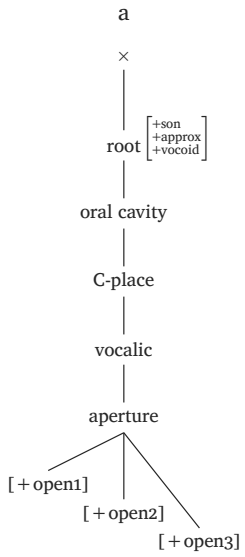


Figure 57: Low vowel

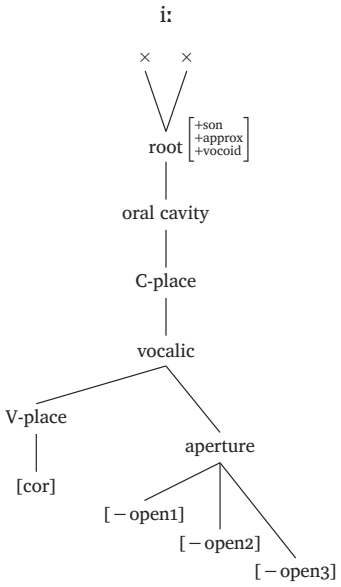


Figure 58: High vowel

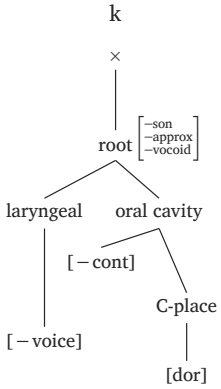


Figure 59: Plosive

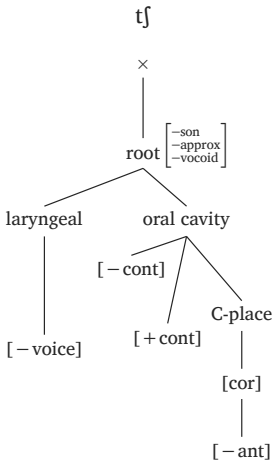


Figure 60: Affricate

```
#geom(ph: "a")
```

```
#geom(ph: "i:")
```

```
#geom(ph: "k")
```

```
#geom(ph: "tS")
```

As you can see, `ph` allows for the most minimal syntax possible, but the function must also ensure that you're not locked to presets. Fortunately, you can easily override any node simply by passing the familiar arguments to the function. For example, let us suppose you want to create the representation for /a/ in a language whose inventory only includes /a i u/. That would only require open 1 (Clements

[5] You can inspect `geom.typ` to see which phonemes are available. The list includes most symbols already available in the `#ipa()` function, including archiphonemes `N` and `T`, as well as placeholders `C` and `V` — just remember you need `\\` before each one of these. Note that you can use `/a/`, `[a]` or `a` as an argument for `ph`, depending on how you want to print the segment at the top of the representation. Finally, the root matrix always has three values by default, which will often be redundant, but you can easily override it by setting your own matrix with the `root` argument.

& Hume, 1995). Simple: `#geom(ph: "a", aperture: (true,))`. This code overrides the `aperture` argument while keeping all other arguments intact for the preset in question.

7.2. Multiple trees and processes

If we can easily represent feature geometric trees, how can we represent *processes* across trees? This is what `#geom-group()` does. It allows for multiple trees to be drawn as a "single object", which is important because we want to be able to draw arrows *across* trees.

Because `#geom-group()` works as a wrapper, you can think of it as a way to "glue" multiple `#geom()` together. As a result, you already know most of the syntax involved. But `#geom-group()` introduces three key arguments: `arrows`, `curved`, and `delinks`. These are all self-explanatory, so let's jump into an example: the spreading of [nasal], shown in Figure 61.

```
#geom-group(
  (ph: "/a/"),
  (ph: "/n/"),
  arrows: (
    (from: "nasal2",
     to: "root1",
     ctrl: (1.1, -1.5)),
    ),
  curved: true, // ← implied
  gap: 1,
)
```

Code 33: Code to create Figure 61

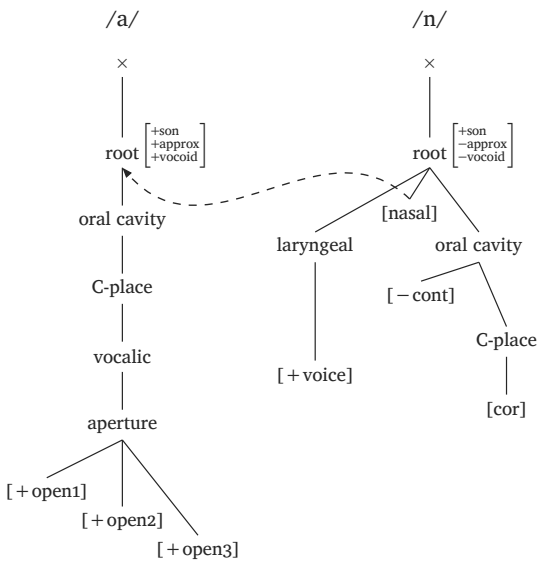


Figure 61: Nasalization example

The way `arrows` works is straightforward: you refer to the nodes you want to connect and add a number that corresponds to the tree. In Figure 61, the arrow goes from `nasal` in tree 2 (hence `nasal2`) to `root` in tree 1 (hence `root1`). Because the argument `curved` is set to `true`, the arrow will try to avoid any obstacles, but there are too many variables involved, so you will often want to adjust the arrow yourself to make sure you like its position. That's why `ctrl` exists inside `arrow`: you modify the **start** and **end** points of the arrow by providing two numbers. Here, `ctrl` is `1.1` and `-1.5`, which means it starts out going up (north) and arrives at its destination from the south (negative number). In most situations, you won't need `ctrl` because the arrow will curve well enough. But it's essential to be able to adjust it in case you don't like its default position (if `ctrl` is provided, `curved: true` is assumed). Finally, `gap` allows you to specify the gap between the trees (defaults to 1.5).

Let us now turn to another example: metaphony. This is a good moment to discuss another argument in both `#geom()` and `#geom-group()`: `model`. By default, these functions use Clements & Hume (1995). But you may not want to assume `aperture` for vowels, for example. If you set `model: sagey`, the function will instead assume the more typical height features (Sagey, 1986). It should be noted that a lot has been said about what feature geometry should look like, and these functions will not do justice

to the long debate in the literature because they introduce too many variables (hence the importance of offering some level of customization to the user). That being said, if you have any suggestions, please open an issue[6] on the package's repository.

Figure 62 shows a typical example of metaphony where /e/ → [i] / ___ C_0 –/u/#. Suppose that we want three trees, where the second tree is a placeholder for any consonant. Here, we also use the `highlight` argument, which allows the user to dim the representation and only highlight certain nodes. The curve is customized again, but you may want to remove the `ctrl` argument to see how the function automatically curves the arrow.

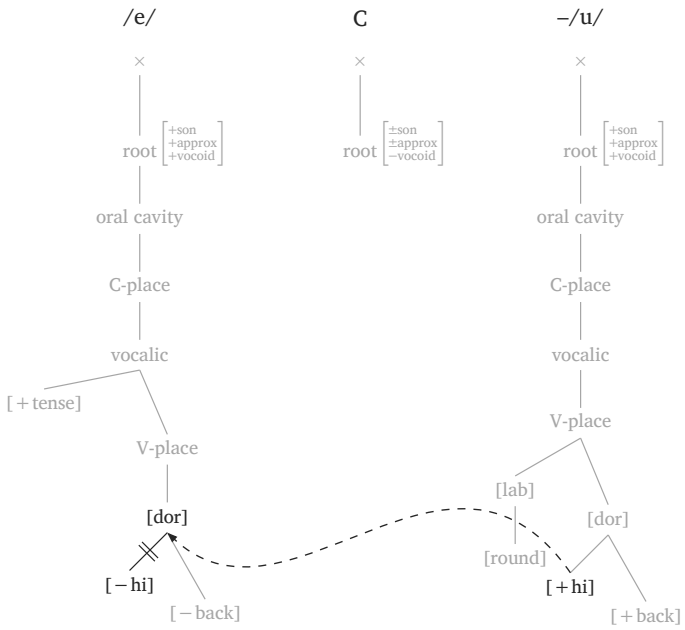


Figure 62: Metaphony using vocalic features in Sagey (1986). `highlight` to focus on process

```
#geom-group(
  (ph: "/e/"),
  (ph: "\\C"),
  (ph: "/u/", prefix: "-"),
  arrows: (
    (from: "high3", to: "dorsal1", ctrl: (2.3, -1.5)),
  ),
  delinks: ("high1",),
  highlight: ("high1", "dorsal1", "high3"),
  gap: 1.5,
  model: "sagey",
)
```

Code 34: Code to create Figure 62

[6]There is a good chance presets will have errors, because they were generated in bulk and it's easy to miss something when proofreading feature specifications. If you find any issues, please flag them so that I can fix them as soon as possible.

As can be seen in Code 34, the syntax of `#geom-group()` is minimal and intuitive, especially if the phonemes needed are pre-built. You should play around with all the available arguments in the function, and test the arrows (both curved and straight) in different scenarios.

8. Optimality theory

Unlike SPE rules, tableaux in optimality theory (OT; Prince & Smolensky (2004, originally circulated in 1993)) are more predictable and constrained. They're also one of the easiest representations to typeset, but if you use LaTeX they're still time-consuming to create (even though you can find tools online to streamline the task). **phonokit** includes two constraint-related functions, the first of which is `#tableau()`, shown in Tableau 1. The function takes six arguments: `input`, `candidates`, `constraints`, `violations`, `winner`, and `dashed-lines`. The accompanying code shown in Tableau 1 provides an example of how each argument works. For example, the `violations` argument requires a nested structure. Likewise, `dashed-lines` requires a comma if you want a given column to have a dashed line. If no dashed lines are needed, you can simply specify `dashed-lines: ()`. Both `winner` and `dashed-lines` count from zero, so `winner: 0` selects the first candidate and `dashed-lines: (0,)` dashes the line after the first constraint. These conventions are the same as those presented for autosegmental representations in Section 6.

```
#tableau(
  input: "/kraTa/",
  candidates: ("kra.Ta", "ka.Ta", "ka.ra.Ta"),
  constraints: ("Max", "Dep", "*Complex"),
  violations: (
    ("", "", "*"),
    ("*!", "", ""),
    ("", "*!", ""),
  ),
  winner: 0, // ← Position of winning cand
  dashed-lines: (0,), // ← Note the comma
  shade: true, // ← true by default
)
```

Code 35: Code to generate Tableau 1

	/kraθa/	Max	Dep	*Complex
☞	[kra.θa]			*
	[ka.θa]	*!		
	[ka.ra.θa]		*!	

Tableau 1: A typical OT tableau

One nice feature of `#tableau()` is that the function automatically shades cells once a fatal violation is entered (`!`). Likewise, it adds the "☞" symbol for the winner, whose position is extracted from the `winner` argument shown in the accompanying code next to Tableau 1. Candidates can also be labeled with letters (a., b., c., …) by setting `letters: true`, as shown in Tableau 2. When letters are used, the ☞ symbol is placed to the left of the winning candidate's letter.

	/kraθa/	MAX	DEP	*COMPLEX
☞ a.	[kra.θa]			*
b.	[ka.θa]	*!		
c.	[ka.ra.θa]		*!	

Tableau 2: A tableau with candidate letters

Additionally, `#tableau()` supports prosodic structures as candidates.[7] You can pass prosodic function calls as content using square brackets, e.g., `[#syllable("mat")]`. This is the recommended approach because it avoids conflicts with the single quote character, which is also used for stress marking in prosodic notation. Tableau 3 shows an example with `#word()` candidates. When passing content directly, you control the scale via the function's own `scale` argument (this is key because prosodic structures are often too large for a tableau). Tableau 3 also shows the `gloss` argument in case more information is needed for the input. This argument requires two strings (orthographic form and translation).

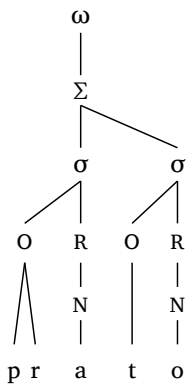
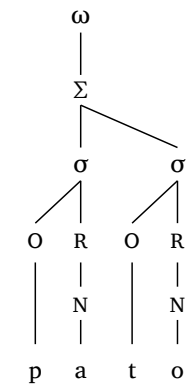
/prato/ *prato* 'plate'	FAITH	*COMPLEX
☞ a. 		*
b. 	*!	

Tableau 3: A tableau with prosodic representation

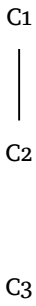
[7] This also applies to other constraint-based functions in the package, discussed later in this manual.

```
#tableau(
  input: "prato",
  candidates: (
  // Notice that candidates aren't strings here
    [#word("('pra.to)", scale: 0.8)],
    [#word("('pa.to)", scale: 0.8)],
  ),
  constraints: ("Faith", "*Complex"),
  violations: (
    ("", "*!"),
    ("", ""),
  ),
  winner: 0,
  letters: true,
  gloss: ("prato", "plate"),
)
```

Code 36: Code to generate Tableau 3

It is often useful to present a ranking using a Hasse diagram. These diagrams can be generated in **phonokit** using the `#hasse()` function. In a nutshell, the function takes tuples with n elements. In the simplest case, $n = 1$, which produces a floating constraint. Hasse diagram 1 shows a basic scenario. The third element in the first tuple indicates the "stratum" in the diagram — this is especially important in more complex cases, which require better control over the vertical position of different constraints. Optional arguments exist to give the user more flexibility (e.g., `scale` and `node-spacing`).

```
#hasse(
   (
     ("C1", "C2", 0),
     ("C3",), // floating constraint
   ),
   node-spacing: 2, // optional
   level-spacing: 2, // optional
   scale: 0.9, // optional
 )
```

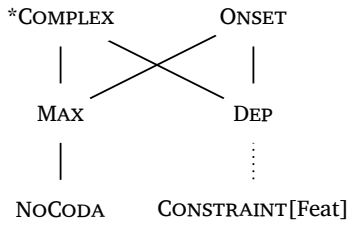


Hasse diagram 1: Basic scenario

A more realistic scenario is shown in Hasse diagram 2. Notice that constraint names are automatically rendered in small caps (except features inside square brackets; see figure). 4-element tuples are also possible, in which case the fourth element allows for different line types between two constraints (`"dashed"` or `"dotted"`).

The top level of a diagram occupies position 0, which includes four partial rankings here. The second level (position 1) has two partial rankings in the diagram in question, namely MAX ≫ NOCODA and DEP ≫ CONSTRAINT. You may want to play around with this position: change it from 1 to 2 and then to 3 to see how this affects the diagram — you may need to manually adjust the `node-spacing` accordingly to avoid constraint overlapping.

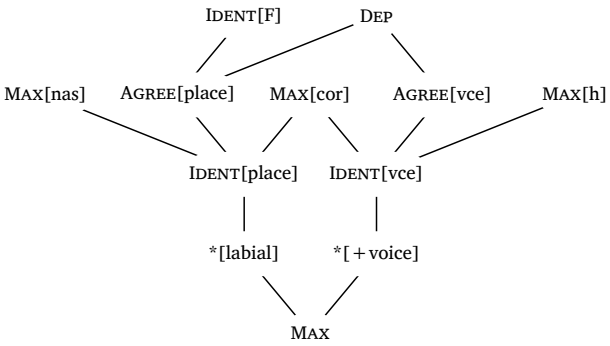
```
#hasse(
  (
    ("*Complex", "Max", 0),
    ("*Complex", "Dep", 0),
    ("Onset", "Max", 0),
    ("Onset", "Dep", 0),
    ("Max", "NoCoda", 1),
    ("Dep", "Constraint[Feat]", 1, "dotted"),
  ),
  node-spacing: 3,
)
```



Hasse diagram 2: Dotted lines and small caps

Finally, let's see a more complex scenario (notice that diagrams are automatically scaled according to their complexity). Hasse diagram 3 is adapted from Lamont (2021).

```
#hasse(
    (
      ("Ident[F]", "Agree[place]", 0),
      ("Dep", "Agree[vce]", 0),
      ("Dep", "Agree[place]", 0),
      ("Max[nas]", "Ident[place]", 1),
      ("Max[cor]", "Ident[place]", 1),
      ("Max[cor]", "Ident[vce]", 1),
      ("Max[h]", "Ident[vce]", 1),
      ("Agree[place]", "Ident[place]", 1),
      ("Agree[vce]", "Ident[vce]", 1),
      ("Ident[place]", "*[labial]", 2),
      ("Ident[vce]", "*[+voice]", 2),
      ("*[labial]", "Max", 3),
      ("*[+voice]", "Max", 3),
    ),
  )
```



Hasse diagram 3: Example from Lamont (2021)

9. Maximum Entropy grammars

Last but not least, the package goes one step further and produces a MaxEnt tableau (Goldwater & Johnson, 2003; Hayes & Wilson, 2008) with the function `#maxent()`. Tableau 1 illustrates a scenario where the data in Tableau 1 are variable, i.e., all candidates in question have a non-zero probability of being observed given a specific input x. The column h_i displays the harmony score of each candidate i, calculated as the weighted sum of all constraint violations. Next, the column e^{-h_i} provides the unnormalized probability, which is the exponential of the negated harmony score (this has also been called the *MaxEnt score*). Finally, the actual predicted probability is shown in column P_i, which is obtained by dividing the unnormalized value of a candidate by $Z(x)$ (the sum of all unnormalized scores). The formal equation for this probability is provided in Equation 1.

$$P(y|x) = \frac{e^{-\sum_{i=1}^{n} w_i C_i(y,x)}}{Z(x)} \tag{1}$$

The function `#maxent()` calculates h_i, e^{-h_i} and $P(y|x)$[8] (shown as P_i in the tableau) automatically given the weights provided. Tableau 1 lists the weights for the constraints in use at the top and prints probability bars at the right margin. These can be turned off with `visualize: false` (see Code 37), but they are printed by default as this can help students quickly visualize probabilities when many candidates are evaluated.

		$w = 2.5$ MAX	$w = 1.8$ DEP	$w = 1$ *COMPLEX	h_i	e^{-h_i}	P_i	
a.	[kra.θa]	0	0	1	1	0.368	0.598	
b.	[ka.ra.θa]	0	1	0	1.8	0.165	0.269	
c.	[ka.θa]	1	0	0	2.5	0.082	0.133	

Tableau 1: A MaxEnt tableau ordered by P_i with `sort: true`

In Code 37, you can see all the necessary arguments for the function `#maxent()`. Like the function `#tableau()` discussed above, the `violations` argument in `#maxent()` requires a nested structure. Everything else is self-explanatory. As expected, the rows and columns will expand as needed for both constraint-based functions.

```
#maxent(
  input: "/kraTa/",
  candidates: ("[kra.Ta]", "[ka.Ta]", "[ka.ra.Ta]"),
  constraints: ("Max", "Dep", "*Complex"),
  weights: (2.5, 1.8, 0.5),
  violations: (
    (0, 0, 1),
    (1, 0, 0),
    (0, 1, 0),
  ),
  visualize: true, // Show probability bars (default)
  sort: true, // Sort candidates from most to least probable
  letters: true // Assigned after sorting when sort: true
)
```

Code 37: Code to generate a MaxEnt tableau

phonokit also has functions for harmonic grammars (`#hg()`) and noisy harmonic grammars (`#nhg()`). These functions are very similar to `#maxent()`, so their syntax will be familiar. The Noisy Harmonic Grammar function derives probabilities by simulating a number of evaluations (by default, 1000) given the constraints and violations provided by the user. It is possible to change the number of simulations and to omit the noise column from the tableau. The noise displayed is extracted from an additional simulation, so it is shown for illustrative purposes. For the most part, the functions discussed in this section are based on conventions in the literature, e.g., Flemming (2021).

Important. `#hg()` and `#nhg()` follow the convention in which violations are **negative** numbers and the candidate with the **highest** harmony wins. This differs from `#maxent()`, where violations

[8] Where y is a given candidate and x is the input.

are positive counts penalizing each candidate ($P^* = e^{-h}$). If you reuse a `violations` array from `#maxent()` in `#hg()` or `#nhg()`, negate the values first — otherwise the harmony scores (and, in `#nhg()`, the simulated probabilities) will be inverted.

```
#hg(
  input: "/kraTa/",
  candidates: ("[kra.Ta]", "[ka.Ta]", "[ka.ra.Ta]"),
  constraints: ("Max", "Dep", "*Complex"),
  weights: (2.5, 1.8, 1),
  violations: (
    (0, 0, -1),
    (-1, 0, 0),
    (0, -1, 0),
  ),
)
```

Code 38: Code to generate an HG tableau (note the negative violations)

10. Numbered examples

One important feature in any phonology paper is an environment for numbered examples. We could simply use the excellent `eggs` package for Typst (Shaldov, 2025), which does a great job for syntax examples. In phonology, however, we sometimes want examples to have arrows to show a process, and this creates an alignment issue when we have multiple instances in a single example or in multiple examples throughout. **phono*kit*** has a function to deal with that: `#ex()`.

Important. To use this function, add this line after importing the package: `#show: ex-rules`

The simplest use of `#ex()` is to wrap content directly. The example number is generated automatically. Use `&` to separate columns, just like in LaTeX tables. The number of columns is inferred from the number of `&` separators in each row.

```
#ex[#ipa("/anba/") & #a-r & #ipa("[amba]")] <phon-single>
```

Code 39: Single-item numbered example

(1) /anba/ → [amba]

For sub-examples, use the list syntax (`-`). Each item is automatically lettered (`a.`, `b.`, etc.). The `labels` argument allows individual sub-examples to be referenced, as in (2a) and (2b). The example as a whole can also be referenced, as in (2).

```
#ex(caption: "A phonology example", labels: (<ex-anba>, <ex-anka>),
  columns: (5em, 2em, 5em))[
  - #ipa("/anba/") & #a-r & #ipa("[amba]")
  - #ipa("/anka/") & #a-r & #ipa("[aNka]")
] <phon-ex>
```

Code 40: Numbered example with sub-examples

(2) a. /anba/ → [amba]
b. /anka/ → [aŋka]

Without `labels`, sub-examples are simply not referenceable individually — only the example as a whole can be labeled and referenced. The `columns` argument is optional and specifies the width of each data column (the number and letter columns are always `2em`). If `columns` is omitted, all data columns default to `auto` width. Using explicit widths guarantees perfect alignment across different examples, as can be seen in (3) and (4).

(3) a. /inbi/ → [imbi]
b. /inki/ → [iŋki]

(4) a. /inbinbi/ → [imbimbi]
b. /inkinki/ → [iŋkiŋki]

Let us now see how strings with ToBI (Section 6.2) can be used in numbered examples. The `title` argument (optional) places a title on the same line as the example number, with the sub-examples below. As usual, you can refer to (5a) and (5b).

```
#ex(
  caption: "Some ToBI examples",
  title: [Autosegmental transcription of
    intonation in English @zsiga2013sounds],
  labels: (<ex-tobi1>, <ex-tobi2>),
)[
  - You're a we#int("*L")rewolf?
    #h(1em)#int("H%", line: false)
  - I'm a wer#int("*H")ewolf.
    #h(1em)#int("L%", line: false)
] <tobi-ex>
```

Code 41: Numbered example with ToBI and `title`

(5) Autosegmental transcription of intonation in English (Zsiga, 2024)

a. *L H%
You're a werewolf?

b. *H L%
I'm a werewolf.

You will notice that `caption` is one of the arguments in `#ex()`. This is not a typical caption (nothing is printed), since this is an example after all. Instead, this caption is there in the (unlikely) event we wish to create a table of contents *only for examples* using the `#outline()` function.[9] The outline below shows what the examples would look like in a table of contents — see Code 42.

List of Examples

(2) A phonology example 45
(3) Another phonology example 45
(4) Yet another phonology example 45
(5) Some ToBI examples 46

```
#outline(
  title: [List of Examples],
  target: figure.where(kind: "linguistic-example"),
)
```

Code 42: Creating a table of examples using `#outline()`

A note on figure spacing. If your document uses a custom `#show figure` rule to add vertical spacing around figures, you may notice misalignment in sub-example labels. This happens because `#subex-label()` uses figures internally, and added spacing will disrupt table cell alignment. To fix this, exclude `linguistic-subexample` from your spacing rule. The present document, for example, does have a rule to add `1em` vertical spacing before and after each figure. To protect `#ex()` from that custom spacing, the preamble should have something along the lines of Code 43.

[9] While it is not very common to have such tables, `#ex()` gives you the possibility. If you want to exclude a given example from the outline, do not use the `caption` argument.

```
#show figure: it => {
  if it.kind == "linguistic-subexample" {
    it
  } else {
    v(1em)
    it
    v(1em)
  }
}
```

Code 43: Excluding linguistic examples from custom figure spacing specifications

11. Final remarks

phonokit is a *very* young project (December 2025). As stated above, its main goal is to quickly generate structures that are frequently used by phonologists when typesetting documents for teaching and research. This means that functions must be **intuitive** — but an intuitive interface cannot sacrifice typographical quality. I hope to have shown that this goal is possible. The package will certainly improve as people start using it in a wider range of scenarios than those presented in this vignette, which represent for the most part the contexts I need myself when preparing documents.

Typst is still in its infancy, and I believe most linguists do not know about it yet (as of January 2026). But as the language expands into linguistics, there is *a lot* of potential for significant advances in our workflows (both in research and teaching). I hope this package will make document preparation quicker and more enjoyable to the phonologists out there.

References

Booij, G. (2012). *The grammar of words: An introduction to linguistic morphology* (3rd ed.). Oxford University Press.

Breit, F. (2025). *linphon: Typesetting phonological rules and feature matrices in Typst*. https://github.com/thatfloflo/typst-linphon

Carvalho, J. B. de. (2017). Deriving sonority from the structure, not the other way round: A Strict CV approach to consonant clusters. *The Linguistic Review*, *34*(4), 589–614.

Chomsky, N., & Halle, M. (1968). *The sound pattern of English*. Harper & Row.

Clements, G. N., & Hume, E. (1995). The internal organization of speech sounds. In J. A. Goldsmith (Ed.), *The Handbook of Phonological Theory: The Handbook of Phonological Theory* (pp. 245–306). Blackwell.

Dreamin, M., & Varner, N. (2024). *Tinymist: An integrated language service for Typst*. https://github.com/Myriad-Dreamin/tinymist

Fejfar, J., & Funke, M. (2024). *CeTZ: A library for drawing with Typst*. https://github.com/cetz-package/cetz

Flemming, E. (2021). Comparing MaxEnt and noisy harmonic grammar. *Glossa: A Journal of General Linguistics*, *6*(1), 1–42. https://doi.org/10.16995/glossa.5775

Garcia, G. D. (2026). *Fonology: Phonological Analysis in R*. https://doi.org/10.5281/zenodo.23087881

Garcia, G. D., & Guzzo, N. B. (2023). A corpus-based approach to map target vowel asymmetry in Brazilian Veneto metaphony. *Italian Journal of Linguistics*, *35*(1), 115–138. https://doi.org/10.26346/1120-2726-205

Goad, H. (2012). sC clusters are (almost always) coda-initial. *Linguistic Review*, *29*(3).

Goldwater, S., & Johnson, M. (2003). Learning OT constraint rankings using a Maximum Entropy model. *Proceedings of the Stockholm Workshop on Variation within Optimality Theory*, 111–120.

Hayes, B. (2009). *Introductory phonology*. John Wiley & Sons.

Hayes, B., & Wilson, C. (2008). A Maximum Entropy Model of Phonotactics and Phonotactic Learning. *Linguistic Inquiry*, *39*(3), 379–440. https://doi.org/10.1162/ling.2008.39.3.379

Labov, W., Ash, S., & Boberg, C. (2008). *The atlas of North American English: Phonetics, phonology and sound change*. Walter de Gruyter.

Ladefoged, P., & Maddieson, I. (1996). *The sounds of the world's languages*. Blackwell.

Lamont, A. (2021). Optimality Theory implements complex functions with simple constraints. *Phonology*, *38*(4), 729–740. https://doi.org/10.1017/s0952675721000361

Mc-Zen. (2025). *Lilaq*. https://typst.app/universe/package/lilaq/

Parker, S. (2011). Sonority. In M. van Oostendorp, C. J. Ewen, E. Hume, & K. Rice (Eds.), *The Blackwell Companion to Phonology: The Blackwell Companion to Phonology* (pp. 1160–1184). Wiley Online Library. https://doi.org/10.1002/9781444335262.wbctp0049

Prince, A., & Smolensky, P. (2004). *Optimality Theory: constraint interaction in Generative Grammar*. Blackwell.

Rei, F. (1996). TIPA: A system for processing phonetic symbols in LATEX. *TUGBoat*.

Sagey, E. (1986). *The Representation of Features and Relations in Nonlinear phonology* [Doctoral dissertation].

Shaldov, A. (2025). *eggs: linguistic examples with minimalist syntax*. https://github.com/retroflexivity/typst-eggs

SIL International. (2025). *Charis*. https://software.sil.org/charis/

Zsiga, E. C. (2024). *The sounds of language: An introduction to phonetics and phonology* (2nd ed.). John Wiley & Sons.

Appendix

A. IPA symbol reference

Plosives															
p	p	b	b	t	t	d	d	\\:t	ʈ	\\:d	ɖ	c	c	\\barredj	ɟ
k	k	g	g	q	q	\\;G	ɢ	?	ʔ	\\barredP	ʡ				
Fricatives															
F	ɸ	B	β	f	f	v	v	T	θ	D	ð	s	s	z	z
S	ʃ	Z	ʒ	\\:s	ʂ	\\:z	ʐ	C	ç	J	ʝ	x	x	G	ɣ
X	χ	K	ʁ	\\barredh	ħ	Q	ʕ	h	h	H	ɦ	\\;H	ʜ	\\barrevglotstop	ʢ
\\textbeltl	ɬ	\\l3	ɮ	*w	ʍ	\\texththeng	ʩ								
Nasals															
m	m	M	ɱ	n	n	\\:n	ɳ	\\nh	ɲ	N	ŋ	\\;N	ɴ		
Approximants & trills															
V	ʋ	*r	ɹ	j	j	\\darkl	ɬ	l	l	L	ʎ	\\:l	ɭ	\\;L	ʟ
r	r	R	ɾ	\\:r	ɽ	\\;R	ʁ	\\mw	ɯ	\\;B	ʙ	\\:R	ɻ	\\turnlonglegr	ɺ
Clicks & implosives															
\\!o	ʘ	\\doublebarpipe	ǂ	||	‖	\\!b	ɓ	\\!d	ɗ	\\!j	ʄ	\\!g	ɠ	\\!G	ɢ
\\textctz	ʑ														
Vowels															
i	i	I	ɪ	y	y	Y	ʏ	1	ɨ	0	ʉ	W	ɯ	u	u
U	ʊ	e	e	\\o	ø	9	ɵ	8	ɵ	7	ɤ	o	o	@	ə
E	ɛ	\\oe	œ	3	ɜ	2	ʌ	O	ɔ	\\ae	æ	\\OE	œ	a	a
5	ɐ	A	ɑ	6	ɒ	\\schwar	ɚ	\\epsilonr	ɝ	\\closeepsilon	ɞ				
Diacritics, suprasegmentals, archiphonemes															
'ta	ˈta	,ta	ˌta	u:	uː	\\~ a	ã	\\r i	i̥	\\v n	n̩	t \\h	t^h	p *	$p^ˀ$
\\t ts	t͡s	k \\labial	k^w	p \\velar	$p^ɣ$	t \\palatal	t^j	\\dental t	t̪	\\C	ɕ	\\V	ʌ	\\N	ɴ

Table 1: IPA Reference Guide (**only main symbols shown**)

B. Symbols in prosodic representations

As of version `0.3.7`, users can decide which symbols are used for prosodic words, feet, syllables and moras. This is accomplished by the argument `symbol`, which requires an array. For consistency, the default symbols are Greek letters: ω, Σ, σ, μ.

For **prosodic words**, the `symbol` array has 3 positions (or 4 for prosodic words using moras). The user can replace the numbers in the code with the symbols/strings of their choice.

```
#word("@.( \\t dZ En).d@",
  symbol: ("1", "2", "3"))
```

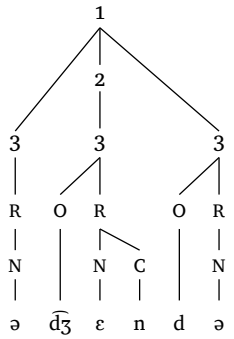


Figure 63: Array for prosodic symbols

For feet, two positions are available (foot and syllable).

```
#foot("\\t dZ En",
  symbol: ("1", "2"))
```

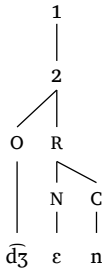


Figure 64: Array for prosodic symbols

For feet with moras, three positions can be changed should the user prefer to alter the default symbol for moras.

```
#foot-mora("\\t dZ En",
  symbol: ("1", "2", "3"))
```

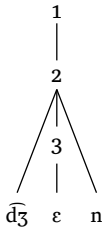


Figure 65: Array for prosodic symbols

Finally, syllables can also be changed.

```
#syllable("\\t dZ En",
  symbol: ("1",))
```



Figure 66: Array for prosodic symbols

C. Wider tableaux

As of version 0.5.3, you can have curly braces, commas, as well as superscript and subscript in input and output forms. Content inside `{ ... }` isn't parsed by the `#ipa()` function. Actual curly braces must be escaped: `\\{ ... \\}`. Constraint names can now have up to 20 characters. The number of characters allowed in constraint names, as well as the maximum number of constraints in a tableau, has been relaxed, so it's now up to the user to decide what a reasonable tableau should look like. See Code 44.

```
#tableau(
  input: "kraTa_{sub}\\, \\s kraTa^{sup}\\, \\s \\{braces\\}",
  candidates: ("long \\s candidate \\s \\A", "candidate \\s \\B", "candidate \\s \\C"),
  constraints: (
    "Constraint-1", "c2", "c3", "c4", "VeryLongConstraint", "c6", "c7", "c8", "c9", "c10", "c11", "c12", "c13", "c14", "c15",
),
  violations: (
    ("", "", "*", "", "", "", "", "", "", "", "*", "", "", "", "*"),
    ("*!", "", "", "", "*", "", "", "", "", "*", "", "", "", "", ""),
    ("", "*!", "", "", "", "", "**", "", "", "", "", "*", "", "", ""),
  ),
  winner: 0,
  dashed-lines: (0,),
  shade: true,
  letters: true,
)
```

Code 44: Code to generate Tableau 4

kraθa$_{sub}$, kraθa^{sup}, {braces}	Constraint-1	c2	c3	c4	VeryLongConstraint	c6	c7	c8	c9	c10	c11	c12	c13	c14	c15
☞ a. long candidate A			*								*				*
b. candidate B	*!				*					*					
c. candidate C		*!					**					*			

Tableau 4: A wide OT tableau

D. How can I use Typst offline?

This vignette assumes that you know about Typst, but you may not be very familiar with it. That's why this section exists. For example, while the online app at Typst.app is very useful and practical, most of us prefer to work offline. **How can you use Typst offline then?**

One of the best IDE options out there is to use VS Code with the extension Tinymist (Dreamin & Varner, 2024) — the extension is therefore available for Positron, which is the successor to RStudio. Tinymist is also available as a plugin for NeoVim users. All these options work extremely well because Tinymist is great, and I haven't had any issues thus far: compilation is instantaneous, and `bib` files also work flawlessly.[10]

If you use Quarto, it is very easy to use **phonokit** with your `qmd` files. You need to first declare `typst` as your format (see Code 45). Then, import the package inside a `typst` code block and you're done. Now you'll be able to use any function you want. Just remember you need the `{=typst}` suffix every time (which you can automate with a simple snippet in Positron, RStudio, etc.; see here).

````
---
title: "A Quarto document"
format: typst
---

```{=typst}
#import "@preview/phonokit:0.6.0": *
```

Now you can use any function you want:

`#syllable("pat"){=typst}`
````

Code 45: Using **phonokit** in a Quarto file

E. How do packages work in Typst?

If you've used R, Python, LaTeX, etc., you are used to installing packages and then importing them. This vignette has imported **phonokit**, of course, which in turn imports CeTZ (Fejfar & Funke, 2024) as a dependency. As you start using Typst, you will notice that it works a bit differently, and this may not be self-evident at first. As seen in Section 1.1, there are basically two ways to load and use a package, both of which require the function `#import` inside your `typ` document — notice that you don't install a package *per se*. The traditional way is to import a package from the official Typst collection/repository, which means adding `#import "@preview/phonokit:0.6.0": *` to your `typ` document if you plan on using **phonokit** (assuming version `0.6.0`). The `@preview` bit indicates that the package comes from Typst's official repository. This is what you should do most of the time. Typst packages are cached once you compile a document with a given package.

Another option is to fork, clone or download a package from GitHub and import its `lib.typ` file instead: `#import "PACKAGE_DIRECTORY/lib.typ": *`. There's only one caveat: Typst restricts imports

[10] Provided that they are clean and do not have any problems regarding fields, repeated entries, etc.

to files within the compilation root and its subdirectories (i.e., you can't load `lib.typ` if the package is in a parent directory or elsewhere in your system). Thus, you may need to use symlinks (this is the same strategy applied to `bib` files if you don't want to have a local copy of your references).

```
# From your working directory:
ln -s PATH_TO_PACKAGE_DIRECTORY package_name
```

Code 46: Creating a symlink to local package

Finally, Typst's repository contains sub-directories to keep track of each version of a given package. When a package is updated, nothing happens to the existing version of the package. Instead, a new directory is added with the updated version. That's why you specify the *version* of a package upon importing: `#import "@preview/phonokit:0.4.0": *`. If you go to Typst's repository on GitHub, you will see that the repository for **phono*kit*** has one sub-directory for each version of the package. This means that you always know which version of a package you are using. And because previous versions are not removed, backwards compatibility is not an issue. If you are used to LaTeX and you have the habit of frequently updating your packages, you will appreciate this, as there's no risk of recompiling your document and running into errors because one of the packages you use has been updated with breaking changes.

F. Exporting representations as images

You may want to use **phono*kit*** without necessarily adopting Typst. The easiest way to do this is to go to Typst.app to create the structure you need using the page set-up shown in Code 47. You can then download the output in PNG format and voilà. You can later add to your LaTeX or Word document — this is similar to using LaTeXiT.

Fortunately, it is also easy to automate this process if you want to do it off-line. First, make sure you install Typst compiler here. Then, create a Typst file with your desired representation. One example is provided in Code 47, where we replicate Tableau 1. In the preamble of the file, notice that we load **phono*kit*** and then adjust our page settings. You will notice that `fill` is set to `none`, which ensures that our resulting PNG file has transparent background. Finally, both `height` and `width` are set to `auto`, which sets the page size dynamically according to the size of the representation you wish to create. We will call this file `maxent.typ`.

```
// Create a file called maxent.typ:
#import "@preview/phonokit:0.6.0": *
#set page(width: auto, height: auto, margin: 0.5em, fill: none)
#maxent(
  input: "kraTa",
  candidates: ("[kra.Ta]", "[ka.Ta]", "[ka.ra.Ta]"),
  constraints: ("Max", "Dep", "*Complex"),
  weights: (2.5, 1.8, 1),
  violations: (
    (0, 0, 1),
    (1, 0, 0),
    (0, 1, 0),
  ),
  visualize: true, // Show probability bars (default)
)
```

Code 47: Example `typ` file to generate a MaxEnt tableau (Tableau 1)

After having created the file in question, simply run the command shown in Code 48 from Terminal. This will generate a PNG file called `maxent.png` with 500 pixels per inch.

```
typst compile --ppi 500 maxent.typ maxent.png
```

Code 48: Compile to PNG in terminal

You could go one step further and use a convenient bash script to take a **phono*kit*** function and generate a PNG figure for any given representation. If you're not familiar with bash scripting, you can use any AI agent to create such a script. An example is provided in Code 49. The script allows you to run the function `phonokit()` from your terminal. Inside the function, you can use any **phono*kit*** function. For example, if you wanted to create a PNG figure for the syllable "ast", you would simply run `phonokit(syllable("ast"))`. The script also gives you the option to set your resolution. By default, the output is saved with a time stamp to your desktop (I'm using Mac OS for reference).

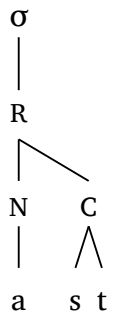


Figure 67: Syllable "ast" generated as a PNG file

```
# NOTE: Function to generate PNG figures using Phonokit
# Adjust Phonokit version and destination as needed
phonokit() {
  local code="$1"
  local ppi="${2:-500}"
  local timestamp=$(date +%Y%m%d_%H%M%S)
  local output="$HOME/Desktop/phonokit_output_${timestamp}.png"
  local tmp=$(mktemp /tmp/phonokit-XXXXXX.typ)

  cat > "$tmp" << EOF
#import "@preview/phonokit:0.6.0": *
#set page(width: auto, height: auto, margin: 0.5em, fill: none)
$code
EOF

  typst compile --ppi "$ppi" "$tmp" "$output"
  rm "$tmp"

  echo "Saved to $output"
}
```

Code 49: Bash script to export PNGs

G. Useful NeoVim keymaps

If you use NeoVim, you may want to set some keymaps for Typst files. Code 50 shows a simplified version of the keymaps I use along with two key plugins: Tinymist (`neovim/nvim-lspconfig`), already mentioned, and `typst-preview` (`chomosuke/typst-preview.nvim`), which allows you to preview `typ` files live in the browser. In a nutshell, these commands could be useful: `\p` to preview a `typ` file on the browser and `\c` to compile the document to a `PDF`. The other two commands listed in Code 50 are also useful: `\i` generates a `PNG` file from the `typ` document (see Section F), and `\s` stops the preview.

```
vim.api.nvim_create_autocmd("FileType", {
  pattern = "typst",
  callback = function()
    vim.keymap.set("n", "<localleader>i", function()
      local file = vim.fn.expand("%")
      local png_name = vim.fn.expand("%:r") .. ".png"
      vim.fn.system("typst compile --format png --ppi 500 " .. vim.fn.shellescape(file))
      if vim.v.shell_error == 0 then
        print("✓ Exported: " .. png_name)
      else
        print("✗ PNG export failed!")
      end
    end, { desc = "Export Typst to PNG (500 ppi)", buffer = true })
    vim.keymap.set("n", "<localleader>p", "<cmd>TypstPreview<cr>", { desc = "Typst
Preview", buffer = true })
    vim.keymap.set("n", "<localleader>s", "<cmd>TypstPreviewStop<cr>", { desc = "Typst
Preview Stop", buffer = true })
    vim.keymap.set("n", "<localleader>c", function()
      local file = vim.fn.expand("%")
      local pdf_name = vim.fn.expand("%:r") .. ".pdf"

      vim.fn.system("typst compile " .. vim.fn.shellescape(file))

      if vim.v.shell_error == 0 then
        print("✓ Compiled: " .. pdf_name)
      else
        print("✗ Compilation failed!")
      end
    end, { desc = "Export Typst to PDF", buffer = true })
  end,
})
```

Code 50: Useful NeoVim keymaps

H. Extras

The `extras.typ` module provides convenience symbols and helper functions.

H.1. Arrows

Function	Symbol	Description
`#a-r`	→	Right arrow
`#a-l`	←	Left arrow
`#a-u`	↑	Up arrow
`#a-d`	↓	Down arrow
`#a-lr`	↔	Bidirectional arrow
`#a-ud`	↕	Vertical bidirectional arrow

`#a-sr`	⇝	Squiggly right arrow
`#a-sl`	⇜	Squiggly left arrow
`#a-r-large`	⟶	Large right arrow with spacing

H.2. Greek symbols

These render upright (non-italicized), unlike math-mode `$sigma$`.

Function	Sym.	Function	Sym.	Function	Sym.
`#alpha`	α	`#mu`	μ	`#tau`	τ
`#beta`	β	`#phi`	φ	`#omega`	ω
`#gamma`	γ	`#pi`	π	`#cap-phi`	Φ
`#delta`	δ	`#sigma`	σ	`#cap-sigma`	Σ
`#lambda`	λ			`#cap-omega`	Ω

H.3. Utilities

Function	Description
`#blank()`	Underline blank for fill-in exercises or SPE rules
	Width is adjustable: `#blank(width: 4em)`
`#extra[ ... ]`	Wraps content in ⟨angle brackets⟩ for extrametricality: `#extra[tion]` → ⟨tion⟩

I. More information, questions, suggestions

If you have any questions, visit `github.com/guilhermegarcia/phonokit`, where you will find all the code for the package. You will also find a discussion page. If you find a bug or typo, or if you'd like to suggest a feature, please open an issue in the repository — this will help improve the package. This is an ongoing project that started in December 2025, so there is *a lot* to be improved.